

2021-3-1

Function literals and value closures proposal for C23

Jens Gustedt
INRIA and ICube, Université de Strasbourg, France

We propose the inclusion of simple lambda expressions into the C standard. We build on a slightly restricted syntax of that feature in C++. In particular, they only have immutable value captures, fully specified parameter types, and, based on N2674, the return type is inferred from **return** statements. This is part of a series of papers for the improvement of type-generic programming in C that has been introduced in N2638. Follow-up papers N2634 and N2635 will extend this feature with **auto** parameter types and lvalue captures, respectively.

Changes:

- v.2. this document, integrating feedback from the WG14 reflector
 - add function literals to the RHS of assignment and cast if the target type is a function pointer
 - make it clear that lambda objects can only be formed by **auto** definitions
 - cleanup of the relationship between lambdas and VM types
 - be more precise on the sequencing of lambda evaluations and function calls
 - affect the attributes of a lambda expression to the lambda value
 - integrate <stdarg.h> and lambdas
 - integrate <setjmp.h> and lambdas
 - integrate lambdas with the rest of the library clause

I. MOTIVATION

In N2638 it is argued that the features presented in this paper are useful in a more general context, namely for the improvement of type-generic programming in C. We will not repeat this argumentation here, but try to motivate the introduction of lambdas as a stand-alone addition to C.

When programming in C we are often confronted with the need of specifying small functional units that

- are to be reused in several places
- are to be passed as argument to another function
- need a fine control of data in- and outflow.

The smallest unit currently is the specification of a function, that is a top-level named entity with identified parameters for input and output. Current C provides several mechanisms to ease the specification of such small functions:

- The possibility to distinguish internal and external linkage via a specification with **static** (or not).
- The possibility to add function definitions to header files and thus to make the definitions and not only the interface declaration available across translation units via the **inline** mechanism.
- The possibility to add additional properties to functions via the attribute mechanism.

All these mechanisms are relatively rigid:

- (1) They require a naming convention for the function.
- (2) They require a specification far away and ahead of the first use.
- (3) They treat all information that is passed from the caller to the function as equally important.

As an example, take the task of specifying a comparison function for strings to `qsort`. There is already such a function, `strcmp`, in the C library that is almost fit for the task, only that its prototype is missing an indirection. The semantically correct comparison function could look something like this:

```
1  int strComp(char* const* a, char* const* b) {  
2      return strcmp(*a, *b);  
3  }
```

Although probably for most existing ABI its call interface could be used as such (if `char* const*` and `void const*` have the same representation) the use of it in the following call is a constraint violation:

```
1  #define NUMEL 256  
2  char* stringArray[NUMEL] = { "hei", "you", ... };  
3  
4  ...  
5  qsort(stringArray, NUMEL, sizeof(char*),  
6        strComp); // mismatch, constraint violation  
7  ...
```

The reflex of some C programmers will perhaps be to paint over this by using a cast:

```
1  ...  
2  qsort(stringArray, NUMEL, sizeof(char*),  
3        (int*)(void const*, void const*)strComp); // UB  
4  ...
```

This does not only make the code barely readable, but also just introduces undefined behavior instead of a constraint violation. On the other hand, on many platforms the behavior of this code may indeed be well defined, because finally the ABI of `strComp` is the right one. Unfortunately there is no way for the programmer to know that for all possible target platforms.

So the “official” strategy in C is to invent yet another wrapper:

```
1  int strCompV(void const* a, void const* b) {  
2      return strComp(a, b);  
3  }  
4  
5  ...  
6  qsort(stringArray, NUMEL, sizeof(char*),  
7        strCompV); // OK  
8  ...
```

This strategy has the disadvantages (1) and (2), but on most platforms it will also miss optimization opportunities:

- Since `strCompV` is specified as a function its address must be unique. The caller cannot inspect `qsort`, it cannot know if `strCompV` and `strComp` must have different addresses. Thus we are forcing the creation of a function that only consists of code duplication.
- If the two functions are found in two different translation units, `strCompV` will just consist of a tail call to `strComp` and thus create a useless indirection for every call within `qsort`.

C++'s lambda feature that we propose to integrate into C allows the following simple specification:

```

1  ...
2  qsort(stringArray, NUMEL, sizeof(char*),
3      [](void const* a, void const* b){
4          return strComp(a, b);
5      });
6  ...

```

By such a specification of a lambda we do not only avoid (1) and (2), but we also leave it to the discretion of the implementation if this produces the a new function with a different address or if the tail call is optimized at the call site and the address of `strComp` is used instead.

Altogether, the improvements that we want to gain with this feature are:

- Similar to compound literals, avoid useless naming conventions for functions with a local scope (anonymous functions).
- Avoid to declare and define small functions far from their use.
- Allow the compiler to reuse functions that have the same functionality and ABI.
- Split interface specifications for such small functions into an invariant part (captures) and into a variable part (parameters).
- Strictly control the in- and outflow of data into specific functional units.
- Provide more optimization opportunities to the compiler, for example better tail call elimination or JIT compilation of code snippets for fixed run-time values.

II. DESIGN CHOICES

II.1. Expression versus function definition

Currently, the C standard imposes to use named callbacks for small functional units that would be used by C library functions such as `atexit` or `qsort`. Where inventing a name is already an unnecessary burden to the programming of small one-use functionalities, the distance between definition and use is a real design problem and can make it difficult to enforce consistency between a callback and a call. Already for the C library itself this is a real problem, because function arguments are even reinterpreted (transiting through `void const*`) by a callback to `qsort`, for example. The situation is even worse, if input data for the function is only implicitly provided by access to global variables as for `atexit`.

Nested functions improve that situation only marginally: definition and use are still dissociated, and access to variables from surrounding scopes can still be used within the local function. In many cases the situation can even be worse than for normal functions, because variables from outside that are accessed by nested functions may have automatic storage duration. Thus, nested functions may access objects that are already dead when they are called, making the behavior of the execution undefined.

For these reasons we opted for an expression syntax referred to as *lambda*. This particular choice notwithstanding we think that it should still be *possible* to name a local functionality if need be, and to reuse it in several places of the same program. Therefore, lambdas still allow to manipulate *lambda values*, the results of a lambda expression, and in particular that these values are assigned to objects of lambda type.

II.2. Capture model

For the possible visibility of types and objects inside the body of a lambda, the simplest is to apply the existing scope model. This is what is chosen here (consistently with C++) for all use of types and objects that do not need an evaluation.

- All visible types can be used, if otherwise permitted, as type name in within **alignof**, **alignas** or **sizeof** expressions, type definitions, generic choice expressions, casts or compound literals, as long as they do not lead to an evaluation of a variably modified type.
- All visible objects can be used within the controlling expression of **_Generic**, within **alignof** expressions, and, if they do not have a variably modified type, within **sizeof** expressions.

In contrast to that and as we have discussed in N2638, there are four possible design choices for the *access* of automatic variables that are visible at the point of the evaluation of a lambda expression. We don't think that there is any "natural" choice among these, but that for a given lambda the choice has to depend on several criteria, some of which are general (such as personal preferences or coding styles) and some of which are special (such as a subsequent modification of the object or optimization questions).

As a consequence, we favor a solution that leaves the principal decision if a capture is a value capture or an lvalue capture to the programmer of the lambda; it is only they who can appreciate the different criteria. For this particular paper, we put the question on how lvalue captures should be handled aside and only introduce value captures.¹ Nevertheless we think that the choice of explicit specification of value captures as provided by C++ lambdas is preferable to the implicit use of value captures for all automatic variables as in Objective C's blocks, or of lvalue captures as for gcc's compound expression or nested functions.²

II.3. Call sequence

As for all papers in this series, we intend not to impose ABI changes to implementations. We chose a specification for a call sequence for lambdas that either uses an existing function call ABI or encapsulates all calls to lambdas within a given translation unit.

For function literals, that is lambdas that have no captures, we impose that they should be convertible to function pointers with the same prototype. It is easy to see that such a lambda can be rewritten to a static function with an auxiliary name which then is used in place of the lambda expression itself.

For closures, that is lambdas with captures, the situation is a bit more complicated. Where some implementations, building for example upon gcc's nested functions, may prefer to use the same calling sequence as for functions, others may want to evaluate captures directly in place and use an extended ABI to call a derived function interface or pass information for the captures implicitly in a special register.

¹Lvalue captures will be proposed in N2635.

²These different possibilities have been discussed in N2638.

Therefore, our proposal just adds lambda values to the possibilities of the postfix expression (LHS) of a function call, and imposes no further restrictions how this feature is to be implemented.

II.4. Interoperability

The fact that objects with lambda type can be defined and may have external linkage, could imply that such lambda objects are made visible between different translation units. If that would be possible, implementations would be forced to extend ABIs with the specification of lambda types, and platforms that have several interoperable implementations would have to agree on such ABI.

To require such an ABI specification would have several disadvantages:

- A cross-implementation effort of for an ABI extension would incur a certain burden for implementations.
- Many different ABI are possible, in particular special cases have a certain potential for optimization. Fixing an ABI too early, forces implementations to give stability guarantees for the interface.

For our proposal here, we expect that most lambda expressions that appear in file scope will be function literals. Since function literals can be converted to function pointers, no special syntax is needed to make their functionalities available to other translation units.

Because there are no objects with automatic storage duration in file scope, the only captures that can be formed in file scope are those that are derived from expressions, and these expression must have a value that can be determined at translation time. We think that it should be possible to define most such captures as lambda-local unmutable objects with static storage duration, and thus, in general such lambdas are better formulated as function literals.

To be accessible in another translation unit a closure expression that is evaluated in block scope, would have to be assigned to a global variable of lambda type. We inhibit this by not specifying a declaration syntax for lambdas. Thereby the only possibility to declare an object of lambda type is to use **auto**, and thus each such declaration must also be a definition such that the full specification of the lambda expression is visible. But then, no translation unit can declare an object of lambda value with external linkage that is not already a definition.

II.5. Invariability

Since lambdas will often concern small functional units, our intent is that implementations use all the means available to optimize them, as long as the security of the execution can be guaranteed. Therefore we will enforce that lambda values, once they are stored in an object, will be known to never change. This will inhibit, e.g, that implementation specific functions or jump targets will change between calls to the same lambda value, or that any lambda value can escape to a context where its originating lambda expression is not known.

II.6. Recursion

Since there is no syntax to forward-declare a lambda and they cannot be assigned, a lambda cannot refer to itself (same lambda value and type), neither directly nor indirectly by calling other functions or lambdas. The only possibility is for function literals, when they

are converted and assigned to function pointers. Such a function pointer can then be used directly or indirectly as any other function pointer, also by the function literal expression that gave rise to its conversion.

```
1  // file scope definition
2  static int (*comp)(void const*, void const*) = 0;
3  ...
4  int main(void) {
5      ...
6      comp = [](void const* A, void const* B){
7          if (something) {
8              return 0;
9          } else {
10             return comp(B, A);
11          }
12      }
13      ...
14 }
```

Such examples for function literals are a bit contrived, and will probably not be very common.

In contrast to that, closures cannot be called recursively because they don't even convert to function pointers. This is a conscious decision for this paper, because we don't want to constrain implementations in the way(s) they reserve the storage that is necessary to hold captures, and how they implement captures in general. For example, closures that return **void** can be implemented relatively simple as-if by adding some small state, an entry label, one return label per call, and some switched or computed **goto** statements.

As a consequence, the maximum storage that is needed for the captures of a given closure can be computed at translation time, and no additional mechanism to handle dynamic storage is necessary.

II.7. Variable argument lists

Although permitted, lambdas with variable argument list are not completely implemented by the major C++ compilers. This seems to indicate that there is not much need for them, and to simplify we have left them out of this specification. If need be, they could be added later with a separate paper.

This notwithstanding, lambdas may have parameters of type **va_list** (`stdarg.h`). This can be useful for small functional units that process variable argument lists of functions.

II.8. Variably modified (VM) types

All VM types, not only VLA, have a hidden state that keeps track of the size or sizes of the current object or the object it points to. Even if such objects may have static storage duration (see e.g 6.7.6.2 p10), their state may have automatic storage duration, and so their use from a lambda is not easily modeled. Therefore their use is completely forbidden with the body of a lambda.

III. SYNTAX AND TERMINOLOGY

For all proposed wording see Section VIII.

III.1. Lambda expressions

Since it is the most flexible and expressive, we propose to adopt C++ syntax for lambdas, 6.5.2.6 p1, as a new form of postfix expression (6.5.2 p1) introducing the terms *lambda expression*, *capture clause*, *capture list*, *capture default*, *value capture*, *capture* and *parameter clause*.

We make some exceptions from that C++ syntax for the scope of this paper:

- (1) We omit the possibility to specify the return type of a lambda. The corresponding C++ syntax

```
-> return-type
```

reuses the `->` token in an unexpected way, and is not strictly necessary if we have **auto** return. If WG14 wishes so, this feature could be added easily in the future as a general function return type syntax.

- (2) We omit the possibility to specify all value captures as mutable. The C++ syntax introduces a keyword, **mutable**, that would be new to C. We don't see enough gain that would justify the introduction of a new keyword.
- (3) For the simplicity of this proposal we omit lvalue captures and lvalue aliases. A follow-up paper, N2635, takes care of lvalue captures. The introduction of lvalue aliases (C++'s references) is not currently planned.
- (4) We omit the possibility for the parameter list to end in a `...` token.

As this syntax leaves the parameter clause as optional, 6.5.2.6 p7 fixes the semantics for this case to be equivalent to an empty parameter list, and also introduces the terminology of *function literal* (no captures) and *closure* (any capture).

Also, 6.5.2.6 p3 introduces a distinction between *explicit captures*, that are captures that are explicitly listed in the capture list, and *implicit captures*, that are automatic variables of a surrounding scope that are caught because the capture clause is **[=]**.

The terminology for *lambda values* and *lambda types* and their *prototype* is introduced with the other type categories in 6.2.5 p20, and then later specified in the clause for lambda expressions, 6.5.2.6 p11.

III.2. Adjustments to other constructs

With the introduction of lambda expressions, functions bodies can now be nested and several standard constructs become ambiguous. Therefore it is necessary to adjust the definitions of these constructs and relate them to the nearest other constructs to which they could refer. This ensures that their use remains unique and well defined, and that no jumps across boundaries of function bodies are introduced.

- For labels we enforce that they are anchored within the nearest function body in which they appear:
 - Function scope as the scope for labels must only extend to the innermost function body in which a label is found and such function scopes are *not* nested (6.2.1 p3).

- Case labels must be found within a corresponding **switch** statement of their innermost function body (6.8.1 p2).
- **continue** and **break** statements must match to a loop or **switch** construct that is found in the innermost function body that contains them (6.8.6.2 p1 and 6.8.6.3 p1).
- A **return** statement also has to be associated to the innermost function body. It has to return a value, if any, according to the type of that function body. Also, if its function body is associated to a lambda, it only has to terminate the corresponding call to the lambda, and not the surrounding function (6.8.6.4 p3).
- We allow function literals to be operand of simple assignment (6.5.16.1 p1) and cast (6.5.4 p2) when the target type is a function pointer.

IV. SEMANTICS

The principal semantic of lambda expressions themselves is described in 6.2.5.6 p7. Namely, it describes how lambda expressions are similar to functions concerning the scope of visibility and the lifetime of captures and parameters.

Captures are handled in two paragraphs, but the main feature is the description of the evaluation that provides values for value captures, 6.5.2.6 p10. It stipulates that their values are determined at the point of evaluation of the lambda expression (basically in order of declaration), that the value undergoes lvalue, array-to-pointer or function-to-pointer conversion if necessary, and that the type of the capture then is the type of the expression *after* that conversion, that is without any qualification or atomic derivation, and, that it gains a **const** qualification. Additionally, we insist that the so-determined value of a value capture will and cannot not change by any means and is the same during all evaluations during all calls to the same lambda value.

Two paragraph, 6.5.2.6 p8 and p9, describe how the two forms of value captures relate and how the type of a value capture is determined. The form without assignment expression is really a short form that evaluates an automatic variable of the surrounding scope of the same name.

The other specifications for lambda expressions are then their use in different contexts.

- Function literals may be converted to function pointers, 6.3.2.1 p5. For these this is easily possible because they have exactly the same functionality as functions: all additional caller information is transferred by arguments to the call. Thus the existing function ABI can be used to call a function literal, and the translator has in fact all information to provide such a call interface.
- As postfix expression within function calls they can take the place that previously only had function pointers, 6.5.2.2. If we would not provide the possibility of captures, the corresponding function literals could all first be converted to function literals (see above) and called then. But since we don't want to impose how lambda-specific capture information is transferred during a call and to guarantee the properties specified in II.3 above, we just add lambdas to the possibilities of the postfix expression that describes the called function.³

³A similar addition for function designators could also be made, see [Gustedt 2016].

V. LIBRARY

The impact on the library clause is relatively small. It mostly concerns an update for the terminology, because the calling context may be a function or a lambda and a callback feature that is referred by a function pointer may indicate an ordinary function or a function literal. Such rectifications concern `<setjmp.h>`, `<signal.h>`, `<stdarg.h>`, `<stdlib.h>` and `<thread.h>`. The impacted library functions or macros are

<code>_Exit</code>	<code>call_once</code>	<code>quick_exit</code>	<code>va_end</code>
<code>at_quick_exit</code>	<code>exit</code>	<code>signal</code>	
<code>atexit</code>	<code>longjmp</code>	<code>thrd_create</code>	
<code>bsearch</code>	<code>qsort</code>	<code>tss_create</code>	

VI. CONSTRAINTS AND REQUIREMENTS

As a general policy, we try to fix as much requirements as possible through constraints, either with specific syntax or explicit constraints. Only if a requirement is not (or hardly) detectable at translation time, or if we want to leave design space to implementations, we formulate it as an imperative, indicating that the behavior then is undefined by the C standard.

- Captures are introduced to handle objects of automatic storage duration, all other categories of objects and functions are to use other mechanisms of access within lambdas. Therefore, we constrain captures to names of objects of automatic storage duration (6.5.2.6 p4) and limit the evaluation of all such objects from a surrounding scope to the initialization of captures. All such evaluations thus take place during the evaluation of the lambda expression itself, not during a subsequent call to the lambda value.
- Unfortunately such a restriction for objects of automatic storage duration is not sufficient to avoid the implicit access of hidden dynamic state from within a lambda. The reason is that there are some rare forms of objects of VM type that have static storage duration, for which even the use in `sizeof` or similar constructs would constitute an evaluation. These are just exotic artifacts in the language without much use cases or justification. We just forbid them by a constraint for this proposal (also 6.5.2.6 p4), but they could be added in later a stage if need be.
- Since an automatic object of array type would evaluate to a pointer type, it would give rise to a capture of a different type than in the surrounding scope. Therefore in 6.5.2.6 p3 and p4 we also add constraints that forbid array types for captures (explicit or implicit) without assignment expression. It is possible to overwrite that constraint by explicitly specifying a capture of the form `id = id`, even if `id` has an array type; within the lambda expression `id` then has pointer type and retrieving the size of the underlying array is not possible.⁴
- Calling a closure needs additional information, namely the transfer of lambda-specific values for captures. In 6.3.2.1 p5 we explicitly call out the fact that converting closures to function pointers is not defined by the text. This would also follow as implicit undefined behavior from the following text, but we found it important to point this out and thereby guide the expectations of programmers.
- A `switch` label should not enable control flow that jumps from the controlling expression of the `switch` into a lambda. The corresponding property is syntactic and can be checked at translation time. Therefore we formulate this as a constraint in 6.8.1 p2.
- Labels should not be used to bypass the calling sequence (capture and parameter instantiation) and jump into a lambda. Therefore we constrain the visibility scope of labels to

⁴Arrays themselves can be accessed as lvalue captures that will be introduced in N2635.

the surrounding function body, 6.2.1 p3. With these constraints, no **goto** statement can be formed that jumps into or out of a lambda or into a different function.

- Similarly, all jump statements other than **return** should never attempt to jump into or out of the nearest enclosing function body. To ensure this we add an explicit constraint as 6.8.6 p2, and in 6.8.6.2 p1 and 6.8.6.3 p1.
- According to II.5 we don't want lambda values to be modified. If they were specified from scratch, this would probably be reflected in both, a constraint and a requirement. But since we want to be able to leave the possibility that lambda values are implemented as function pointers (in particular for function literals) we cannot make this a requirement. Therefore, we only introduce a requirement (6.5.2.6 p11 last sentence) and recommended practice for applications to use a **const** qualification and for implementations to diagnose modifications when possible (6.5.2.6 p12).
- There is no direct syntax to declare lambda types, and so objects of lambda type can only be declared (and defined) through type inference. The necessary adjustments to that feature are integrated to the constraints of 6.7.10 p4.

VII. QUESTIONS FOR WG14

- (1) Does WG14 want the lambda feature for C23 along the lines of N2675?
- (2) Does WG14 want to integrate the changes as specified in N2675 into C23?

Acknowledgments

Many thanks go to and to many other WG14ners for the discussions, especially to Joseph Myers for his very detailed review and feedback,

References

- Jens Gustedt. 2016. *The register overhaul*. Technical Report N2067. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2067.pdf>.
- Jens Gustedt. 2021a. *Function literals and value closures*. Technical Report N2675. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2675.pdf>.
- Jens Gustedt. 2021b. *Improve type generic programming*. Technical Report N2638. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2638.pdf>.
- Jens Gustedt. 2021c. *Lvalue closures*. Technical Report N2635. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2635.pdf>.
- Jens Gustedt. 2021d. *Type-generic lambdas*. Technical Report N2634. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2634.pdf>.
- Jens Gustedt. 2021e. *Type inference for variable definitions and function return*. Technical Report N2674. ISO. available at <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2674.pdf>.

VIII. PROPOSED WORDING

The proposed text is given as diff against N2674.

- Additions to the text are marked as shown.
- Deletions of text are marked as ~~shown~~.

6. Language

6.1 Notation

- 1 In the syntax notation used in this clause, syntactic categories (nonterminals) are indicated by *italic type*, and literal words and character set members (terminals) by **bold type**. A colon (:) following a nonterminal introduces its definition. Alternative definitions are listed on separate lines, except when prefaced by the words “one of”. An optional symbol is indicated by the subscript “opt”, so that

{ *expression*_{opt} }

indicates an optional expression enclosed in braces.

- 2 When syntactic categories are referred to in the main text, they are not italicized and words are separated by spaces instead of hyphens.
- 3 A summary of the language syntax is given in Annex A.

6.2 Concepts

6.2.1 Scopes of identifiers

- 1 An identifier can denote an object; a function; a tag or a member of a structure, union, or enumeration; a typedef name; a label name; a macro name; or a macro parameter. The same identifier can denote different entities at different points in the program. A member of an enumeration is called an *enumeration constant*. Macro names and macro parameters are not considered further here, because prior to the semantic phase of program translation any occurrences of macro names in the source file are replaced by the preprocessing token sequences that constitute their macro definitions.
- 2 For each different entity that an identifier designates, the identifier is *visible* (i.e., can be used) only within a region of program text called its *scope*. Different entities designated by the same identifier either have different scopes, or are in different name spaces. There are four kinds of scopes: function, file, block, and function prototype. (A *function prototype* is a declaration of a function that declares the types of its parameters.)
- 3 A label name is the only kind of identifier that has *function scope*. It can be used (in a **goto** statement) ~~anywhere~~ in the function body in which it appears, and is declared implicitly by its syntactic appearance (followed by a : and a statement). Each function body has a function scope that is separate from the function scope of any other function body. In particular, a label is visible in exactly one function scope (the innermost function body in which it appears) and distinct function bodies may use the same identifier to designate different labels.²⁹⁾
- 4 Every other identifier has scope determined by the placement of its declaration (in a declarator or type specifier). If the declarator or type specifier that declares the identifier appears outside of any block or list of parameters, the identifier has *file scope*, which terminates at the end of the translation unit. If the declarator or type specifier that declares the identifier appears inside a block or within the list of parameter declarations in a function definition, the identifier has *block scope*, which terminates at the end of the associated block. If the declarator or type specifier that declares the identifier appears within the list of parameter declarations in a function prototype (not part of a function definition), the identifier has *function prototype scope*, which terminates at the end of the function declarator.³⁰⁾ If an identifier designates two different entities in the same name space, the scopes might overlap. If so, the scope of one entity (the *inner scope*) will end strictly before the scope of the other entity (the *outer scope*). Within the inner scope, the identifier designates the entity declared in the inner scope; the entity declared in the outer scope is *hidden* (and not visible) within the inner scope.

²⁹⁾ As a consequence, it is not possible to specify a **goto** statement that jumps into or out of a lambda or into another function.

³⁰⁾ Identifiers that are defined in the parameter list of a lambda expression do not have prototype scope, but a scope that comprises the whole body of the lambda.

- A *structure type* describes a sequentially allocated nonempty set of member objects (and, in certain circumstances, an incomplete array), each of which has an optionally specified name and possibly distinct type.
- A *union type* describes an overlapping nonempty set of member objects, each of which has an optionally specified name and possibly distinct type.
- A *function type* describes a function with specified return type. A function type is characterized by its return type and the number and types of its parameters. A function type is said to be derived from its return type, and if its return type is *T*, the function type is sometimes called “function returning *T*”. The construction of a function type from a return type is called “function type derivation”.
- A *lambda type* is a complete object type that describes the value of a lambda expression. A lambda type is characterized but not determined by a return type that is inferred from the function body of the lambda expression, and by the number, order, and type of parameters that are expected for function calls. The function type that has the same return type and list of parameter types as the lambda is called the *prototype* of the lambda. A lambda type has no syntax derivation.⁵⁰⁾ Objects of such a type shall only be defined as a capture (of another lambda expression) or by an underspecified declaration for which the lambda type is inferred; such an object shall not be modified.
- A *pointer type* may be derived from a function type or an object type, called the *referenced type*. A pointer type describes an object whose value provides a reference to an entity of the referenced type. A pointer type derived from the referenced type *T* is sometimes called “pointer to *T*”. The construction of a pointer type from a referenced type is called “pointer type derivation”. A pointer type is a complete object type.
- An *atomic type* describes the type designated by the construct `_Atomic(type-name)`. (Atomic types are a conditional feature that implementations need not support; see 6.10.8.3.)

These methods of constructing derived types can be applied recursively.

- 21 Arithmetic types and pointer types are collectively called *scalar types*. Array and structure types are collectively called *aggregate types*.⁵¹⁾
- 22 An array type of unknown size is an incomplete type. It is completed, for an identifier of that type, by specifying the size in a later declaration (with internal or external linkage). A structure or union type of unknown content (as described in 6.7.2.3) is an incomplete type. It is completed, for all declarations of that type, by declaring the same structure or union tag with its defining content later in the same scope.
- 23 A type has *known constant size* if the type is not incomplete and is not a variable length array type.
- 24 Array, function, and pointer types are collectively called *derived declarator types*. A *declarator type derivation* from a type *T* is the construction of a derived declarator type from *T* by the application of an array-type, a function-type, or a pointer-type derivation to *T*.
- 25 A type is characterized by its *type category*, which is either the outermost derivation of a derived type (as noted above in the construction of derived types), or the type itself if the type consists of no derived types.
- 26 Any type so far mentioned is an *unqualified type*. Each unqualified type has several *qualified versions* of its type,⁵²⁾ corresponding to the combinations of one, two, or all three of the **const**, **volatile**, and **restrict** qualifiers. The qualified or unqualified versions of a type are distinct types that

⁵⁰⁾ Not even a **typeof** type specifier with lambda type can be formed. So there is no syntax to make a lambda type a choice in a generic selection other than **default**

⁵¹⁾ Note that aggregate type does not include union type because an object with union type can only contain one member at a time.

⁵²⁾ See 6.7.3 regarding qualified array and function types.

lvalue that does not have array type is converted to the value stored in the designated object (and is no longer an lvalue); this is called *lvalue conversion*. If the lvalue has qualified type, the value has the unqualified version of the type of the lvalue; additionally, if the lvalue has atomic type, the value has the non-atomic version of the type of the lvalue; otherwise, the value has the type of the lvalue. If the lvalue has an incomplete type and does not have array type, the behavior is undefined. If the lvalue designates an object of automatic storage duration that could have been declared with the **register** storage class (never had its address taken), and that object is uninitialized (not declared with an initializer and no assignment to it has been performed prior to use), the behavior is undefined.

- 3 Except when it is the operand of the **typeof** specifier, the unary **sizeof** operator, or the unary & operator, or is a string literal used to initialize an array, an expression that has type “array of *type*” is converted to an expression with type “pointer to *type*” that points to the initial element of the array object and is not an lvalue. If the array object has register storage class, the behavior is undefined.
- 4 A *function designator* is an expression that has function type. Except when it is the operand of the **typeof** specifier, the **sizeof** operator,⁷¹⁾ or the unary & operator, a function designator with type “function returning *type*” is converted to an expression that has type “pointer to function returning *type*”.
- 5 Closures shall not be converted to any other object type. A function literal with a type “lambda with prototype *type*” can be converted implicitly or explicitly to an expression that has type “pointer to *type*”⁷²⁾ The function pointer value behaves as if a function with internal linkage with the appropriate prototype, a unique name, and the same function body as for λ had been specified in the translation unit and the function pointer had been formed by function-to-pointer conversion of that function. The only difference is that the function pointer needs not necessarily to be distinct from any other compatible function pointer that provides the same observable behavior.

Forward references: [lambda expressions \(6.5.2.6\)](#) address and indirection operators (6.5.3.2), assignment operators (6.5.16), common definitions <stddef.h> (7.19), **typeof** specifier 6.7.9, initialization (6.7.10), postfix increment and decrement operators (6.5.2.4), prefix increment and decrement operators (6.5.3.1), the **sizeof** and **_Alignof** operators (6.5.3.4), structure and union members (6.5.2.3).

6.3.2.2 void

- 1 The (nonexistent) value of a *void expression* (an expression that has type **void**) shall not be used in any way, and implicit or explicit conversions (except to **void**) shall not be applied to such an expression. If an expression of any other type is evaluated as a void expression, its value or designator is discarded. (A void expression is evaluated for its side effects.)

6.3.2.3 Pointers

- 1 A pointer to **void** may be converted to or from a pointer to any object type. A pointer to any object type may be converted to a pointer to **void** and back again; the result shall compare equal to the original pointer.
- 2 For any qualifier *q*, a pointer to a non-*q*-qualified type may be converted to a pointer to the *q*-qualified version of the type; the values stored in the original and converted pointers shall compare equal.
- 3 An integer constant expression with the value 0, or such an expression cast to type **void ***, is called a *null pointer constant*.⁷³⁾ If a null pointer constant is converted to a pointer type, the resulting pointer, called a *null pointer*, is guaranteed to compare unequal to a pointer to any object or function.
- 4 Conversion of a null pointer to another pointer type yields a null pointer of that type. Any two null pointers shall compare equal.
- 5 An integer may be converted to any pointer type. Except as previously specified, the result is implementation-defined, might not be correctly aligned, might not point to an entity of the referenced

⁷¹⁾Because this conversion does not occur, the operand of the **sizeof** operator remains a function designator and violates the constraints in 6.5.3.4.

⁷²⁾It follows that lambdas of different type cannot be assigned to each other. Thus, in the conversion of a function literal to a function pointer, the prototype of the originating lambda expression can be assumed to be known, and a diagnostic can be issued if the prototypes do not agree.

⁷³⁾The macro **NULL** is defined in <stddef.h> (and other headers) as a null pointer constant; see 7.19.

default : *assignment-expression*

Constraints

- 2 A generic selection shall have no more than one **default** generic association. The type name in a generic association shall specify a complete object type other than a variably modified type. No two generic associations in the same generic selection shall specify compatible types. The type of the controlling expression is the type of the expression as if it had undergone an lvalue conversion,¹⁰⁰⁾ array to pointer conversion, or function to pointer conversion. That type shall be compatible with at most one of the types named in the generic association list. If a generic selection has no **default** generic association, its controlling expression shall have type compatible with exactly one of the types named in its generic association list.

Semantics

- 3 The controlling expression of a generic selection is not evaluated. If a generic selection has a generic association with a type name that is compatible with the type of the controlling expression, then the result expression of the generic selection is the expression in that generic association. Otherwise, the result expression of the generic selection is the expression in the **default** generic association. None of the expressions from any other generic association of the generic selection is evaluated.
- 4 The type and value of a generic selection are identical to those of its result expression. It is an lvalue, a function designator, or a void expression if its result expression is, respectively, an lvalue, a function designator, or a void expression. A generic selection that is the operand of a **typeof** specification behaves as if the selected assignment expression had been the operand.
- 5 **EXAMPLE** The **cbirt** type-generic macro could be implemented as follows:

```
#define cbirt(X) _Generic((X),
                        long double: cbirtl,
                        default: cbirt,
                        float: cbirtf
                        )(X)
```

6.5.2 Postfix operators

Syntax

- 1 *postfix-expression*:
 - primary-expression*
 - postfix-expression* [*expression*]
 - postfix-expression* (*argument-expression-list*_{opt})
 - postfix-expression* . *identifier*
 - postfix-expression* -> *identifier*
 - postfix-expression* ++
 - postfix-expression* -
 - (*type-name*) { *initializer-list* }
 - (*type-name*) { *initializer-list* , }
 - lambda-expression*
- argument-expression-list*:
- assignment-expression*
 - argument-expression-list* , *assignment-expression*

¹⁰⁰⁾ An lvalue conversion drops type qualifiers.

6.5.2.1 Array subscripting

Constraints

- 1 One of the expressions shall have type “pointer to complete object *type*”, the other expression shall have integer type, and the result has type “*type*”.

Semantics

- 2 A postfix expression followed by an expression in square brackets [] is a subscripted designation of an element of an array object. The definition of the subscript operator [] is that **E1[E2]** is identical to $(*((\mathbf{E1})+(\mathbf{E2})))$. Because of the conversion rules that apply to the binary + operator, if **E1** is an array object (equivalently, a pointer to the initial element of an array object) and **E2** is an integer, **E1[E2]** designates the **E2**-th element of **E1** (counting from zero).
- 3 Successive subscript operators designate an element of a multidimensional array object. If **E** is an n -dimensional array ($n \geq 2$) with dimensions $i \times j \times \cdots \times k$, then **E** (used as other than an lvalue) is converted to a pointer to an $(n - 1)$ -dimensional array with dimensions $j \times \cdots \times k$. If the unary * operator is applied to this pointer explicitly, or implicitly as a result of subscripting, the result is the referenced $(n - 1)$ -dimensional array, which itself is converted into a pointer if used as other than an lvalue. It follows from this that arrays are stored in row-major order (last subscript varies fastest).
- 4 **EXAMPLE** Consider the array object defined by the declaration

```
int x[3][5];
```

Here **x**

is a 3×5 array of

int s; more precisely, **x** is an array of three element objects, each of which is an array of five **int** s. In the expression **x[i]**, which is equivalent to $(*((\mathbf{x})+(\mathbf{i})))$, **x** is first converted to a pointer to the initial array of five **int** s. Then **i** is adjusted according to the type of **x**, which conceptually entails multiplying **i** by the size of the object to which the pointer points, namely an array of five **int** objects. The results are added and indirection is applied to yield an array of five **int** s. When used in the expression **x[i][j]**, that array is in turn converted to a pointer to the first of the **int** s, so **x[i][j]** yields an **int**.

Forward references: additive operators (6.5.6), address and indirection operators (6.5.3.2), array declarators (6.7.6.2).

6.5.2.2 Function calls

Constraints

- 1 The ~~expression that denotes the called function~~ postfix expression¹⁰¹⁾ shall have ~~type~~ lambda type or pointer to function type, returning **void** or returning a complete object type other than an array type.
- 2 If the ~~expression that denotes the called function has a type that~~ postfix expression is a lambda or if the type of the function includes a prototype, the number of arguments shall agree with the number of parameters of the function or lambda type. Each argument shall have a type such that its value may be assigned to an object with the unqualified version of the type of its corresponding parameter.

Semantics

- 3 A postfix expression followed by parentheses () containing a possibly empty, comma-separated list of expressions is a function call. The postfix expression denotes the called function or lambda. The list of expressions specifies the arguments to the function or lambda.
- 4 An argument may be an expression of any complete object type. In preparing for the call to a function, the arguments are evaluated, and each parameter is assigned the value of the corresponding argument.¹⁰²⁾
- 5 If the expression that denotes the called function has lambda type or type pointer to function returning an object type, the function call expression has the same type as that object type, and has the value determined as specified in 6.8.6.4. Otherwise, the function call has type **void**.

¹⁰¹⁾Most often, this is the result of converting an identifier that is a function designator.

¹⁰²⁾A function or lambda can change the values of its parameters, but these changes cannot affect the values of the arguments. On the other hand, it is possible to pass a pointer to an object, and the function or lambda can then change the value of the object pointed to. A parameter declared to have array or function type is adjusted to have a pointer type as described in 6.9.1.

- 6 If the expression that denotes the called function has a type that does not include a prototype, the integer promotions are performed on each argument, and arguments that have type **float** are promoted to **double**. These are called the *default argument promotions*. If the number of arguments does not equal the number of parameters, the behavior is undefined. If the function is defined with a type that includes a prototype, and either the prototype ends with an ellipsis (, ...) or the types of the arguments after promotion are not compatible with the types of the parameters, the behavior is undefined. If the function is defined with a type that does not include a prototype, and the types of the arguments after promotion are not compatible with those of the parameters after promotion, the behavior is undefined, except for the following cases:
 - one promoted type is a signed integer type, the other promoted type is the corresponding unsigned integer type, and the value is representable in both types;
 - both types are pointers to qualified or unqualified versions of a character type or **void**.
- 7 If the expression that denotes the called function [is a lambda or is a function](#) has a type that does not include a prototype, the arguments are implicitly converted, as if by assignment, to the types of the corresponding parameters, taking the type of each parameter to be the unqualified version of its declared type. The ellipsis notation in a function prototype declarator causes argument type conversion to stop after the last declared parameter. The default argument promotions are performed on trailing arguments.
- 8 No other conversions are performed implicitly; in particular, the number and types of arguments are not compared with those of the parameters in a function definition that does not include a function prototype declarator.
- 9 If the [lambda or](#) function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called [lambda or](#) function, the behavior is undefined.
- 10 There is a sequence point after the evaluations of the function designator and the actual arguments but before the actual call. Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or after the execution of the body of the called function [or lambda](#) is indeterminately sequenced with respect to the execution of the called function.¹⁰³⁾
- 11 Recursive function calls shall be permitted, both directly and indirectly through any chain of other functions [or lambdas](#).
- 12 **EXAMPLE** In the function call

```
(*pf[f1()]) (f2(), f3() + f4())
```

the functions **f1**, **f2**, **f3**, and **f4** can be called in any order. All side effects have to be completed before the function pointed to by **pf[f1()]** is called.

Forward references: function declarators (including prototypes) (6.7.6.3), function definitions (6.9.1), the **return** statement (6.8.6.4), simple assignment (6.5.16.1).

6.5.2.3 Structure and union members

Constraints

- 1 The first operand of the . operator shall have an atomic, qualified, or unqualified structure or union type, and the second operand shall name a member of that type.
- 2 The first operand of the-> operator shall have type “pointer to atomic, qualified, or unqualified structure” or “pointer to atomic, qualified, or unqualified union”, and the second operand shall name a member of the type pointed to.

Semantics

- 3 A postfix expression followed by the . operator and an identifier designates a member of a structure or union object. The value is that of the named member,¹⁰⁴⁾ and is an lvalue if the first expression is

¹⁰³⁾In other words, function executions do not “interleave” with each other.

¹⁰⁴⁾If the member used to read the contents of a union object is not the same as the member last used to store a value in the object, the appropriate part of the object representation of the value is reinterpreted as an object representation in the new

The first always has static storage duration and has type array of **char**, but need not be modifiable; the last two have automatic storage duration when they occur within the body of a function, and the first of these two is modifiable.

- 13 **EXAMPLE 6** Like string literals, const-qualified compound literals can be placed into read-only memory and can even be shared. For example,

```
(const char []){"abc"} == "abc"
```

might yield 1 if the literals' storage is shared.

- 14 **EXAMPLE 7** Since compound literals are unnamed, a single compound literal cannot specify a circularly linked object. For example, there is no way to write a self-referential compound literal that could be used as the function argument in place of the named object `endless_zeros` below:

```
struct int_list { int car; struct int_list *cdr; };
struct int_list endless_zeros = {0, &endless_zeros};
eval(endless_zeros);
```

- 15 **EXAMPLE 8** Each compound literal creates only a single object in a given scope:

```
struct s { int i; };

int f (void)
{
    struct s *p = 0, *q;
    int j = 0;

    again:
        q = p, p = &((struct s){ j++ });
        if (j < 2) goto again;

    return p == q && q->i == 1;
}
```

The function `f()` always returns the value 1.

- 16 Note that if an iteration statement were used instead of an explicit **goto** and a labeled statement, the lifetime of the unnamed object would be the body of the loop only, and on entry next time around `p` would have an indeterminate value, which would result in undefined behavior.

Forward references: type names (6.7.7), initialization (6.7.10).

6.5.2.6 Lambda expressions

Syntax

- 1 *lambda-expression:*
~~~~~ *capture-clause* *parameter-clause*<sub>opt</sub> *attribute-specifier-sequence*<sub>opt</sub> *function-body*

*capture-clause:*

```
~~~~~ [ ]
~~~~~ [ capture-list ]
~~~~~ [ capture-default ]
```

*capture-list:*

```
~~~~~ value-capture
~~~~~ capture-list , value-capture
```

*capture-default:*

```
~~~~~ =
```

*value-capture:*

```
~~~~~ capture
~~~~~ capture = assignment-expression
```

*capture:*  
~~~~~ *identifier*

parameter-clause:
~~~~~ ( *parameter-list*<sub>opt</sub> )

### Constraints

- 2 A capture that is listed in the capture list is an *explicit capture* . If the capture clause is [=], *id* is the name of an object with automatic storage duration in a surrounding scope that is not an array, *id* is used within the function body of the lambda without redeclaration and *id* is not a parameter, the effect is as if a capture list had been specified with *id* as a member. Such a capture is an *implicit capture* .
- 3 Captures without assignment expression shall be names of complete objects with automatic storage duration in a scope surrounding the lambda expression that do not have array type and that are visible at the point of evaluation of the lambda expression. An identifier shall appear at most once; either as an explicit capture or as a parameter name in the parameter type list.
- 4 Within the lambda expression, identifiers (including explicit and implicit captures, and parameters of the lambda) shall be used according to the usual scoping rules, but outside the assignment expression of a value capture the following exceptions apply to identifiers that are declared in a scope that strictly includes the lambda expression:
  - Objects or type definitions with VM type shall not be used.
  - Objects with automatic storage duration shall not be evaluated.<sup>111)</sup>
- 5 The function body shall be such that a return type *type* according to the rules in 6.8.6.4 can be inferred.

### Semantics

- 6 The optional attribute specifier sequence in a lambda expression appertains to the resulting lambda value. If the parameter clause is omitted, a clause of the form ( ) is assumed. A lambda expression without any capture is called a *function literal expression* , otherwise it is called a *closure expression* . A lambda value originating from a function literal expression is called a *function literal* , otherwise it is called a *closure* .
- 7 Similar to a function definition, a lambda expression forms a single block scope that comprises its capture clause, its parameter clause and its function body. Each explicit capture and parameter has a scope of visibility that starts immediately after its definition is completed and extends to the end of the function body. The scope of visibility of implicit captures is the function body. In particular, captures and parameters are visible throughout the whole function body, unless they are redeclared in a depending block within that function body. Captures and parameters have automatic storage duration; in each function call to the formed lambda value, a new instance of each capture and parameter is created and initialized in order of declaration and has a lifetime until the end of the call, only that the addresses of captures are not necessarily unique.
- 8 If a capture *id* is defined without an assignment expression, the assignment expression is assumed to be *id* itself, referring to the object of automatic storage duration of the surrounding scope that exists according to the constraints.<sup>112)</sup>
- 9 The implicit or explicit assignment expression *E* in the definition of a value capture determines a value *E*<sub>0</sub> with type *T*<sub>0</sub>, which is *E* after possible lvalue, array-to-pointer or function-to-pointer

<sup>111)</sup> Identifiers of visible automatic objects that are not captures and that do not have a VM type, may still be used if they are not evaluated, for example in **sizeof** expressions, in **typeof** specifiers (if they are not lambdas themselves) or as controlling expression of a generic primary expression.

<sup>112)</sup> The evaluation in rules in the next paragraph then stipulates that it is evaluated at the point of evaluation of the lambda expression, and that within the body of the lambda an unmutable **auto** object of the same name, value and type is made accessible.

conversion. The type of the capture is  $T_0$  **const** and its value is  $E_0$  for all evaluations in all function calls to the lambda value. If, within the function body, the address of the capture id or one of its members is taken, either explicitly by applying a unary & operator or by an array to pointer conversion,<sup>113)</sup> and that address is used to modify the underlying object, the behavior is undefined.

- 10 The evaluation of the explicit or implicit assignment expressions of value captures takes place during each evaluation of the lambda expression. The evaluation of assignment expressions for explicit value captures is sequenced in order of declaration; an earlier capture may occur within an assignment expression of a later one. The objects of automatic storage duration corresponding to implicit value captures are evaluated unsequenced among each other. The evaluation of a lambda expression is sequenced before any use of the resulting lambda value. For each call to a lambda value, explicit value captures (with type and value as determined during the evaluation of the lambda expression) and then parameter types and values are determined in order of declaration. Explicit value captures and earlier parameters may occur within the declaration of a later one.
- 11 For each lambda expression, the return type *type* is inferred as indicated in the constraints. A lambda expression  $\lambda$  has an unspecified lambda type  $L$  that is the same for every evaluation of  $\lambda$ . As a result of the expression, a value of type  $L$  is formed that identifies  $\lambda$  and the specific set of values of the identifiers in the capture clause for the evaluation, if any. This is called a *lambda value*. It is unspecified, whether two lambda expressions  $\lambda$  and  $\kappa$  share the same lambda type even if they are lexically equal but appear at different points of the program. Objects of lambda type shall not be modified.

### Recommended practice

- 12 To avoid their accidental modification, it is recommended that declarations of lambda type objects are **const** qualified. Whenever possible, implementations are encouraged to diagnose any attempt to modify a lambda type object.
- 13 **EXAMPLE 1** The usual scoping rules extend to lambda expressions; the concept of captures only restricts which identifiers may be evaluated or not.

```
#include <stdio.h>
static long var;
int main(void) {
    [ ](void){ printf("%ld\n", var); }(); // valid, prints 0
    [var](void){ printf("%ld\n", var); }(); // invalid, var is static

    int var = 5;

    [var](void){ printf("%d\n", var); }(); // valid, prints 5
    [ ](void){ printf("%d\n", var); }(); // invalid
    [var](void){ printf("%zu\n", sizeof var); }(); // valid, prints sizeof(int)
    [ ](void){ printf("%zu\n", sizeof var); }(); // valid, prints sizeof(int)
    [ ](void){ extern long var; printf("%ld\n", var); }(); // valid, prints 0
}
```

- 14 **EXAMPLE 2** The following uses a function literal as a comparison function argument for **qsort**.

```
#define SORTFUNC(TYPE) [](size_t nmemb, TYPE A[nmemb]) { \
    qsort(A, nmemb, sizeof(A[0]), \
        [](void const* x, void const* y){ /* comparison lambda */ \
            TYPE X = *(TYPE const*)x; \
            TYPE Y = *(TYPE const*)y; \
            return (X < Y) ? -1 : ((X > Y) ? 1 : 0); /* return of type int */ \
        } \
    ); \
    return A; \
}
```

<sup>113)</sup> The capture does not have array type, but if it has a union or structure type, one of its members may have such a type.

```

}
...
long C[5] = { 4, 3, 2, 1, 0, };
SORTFUNC(long)(5, C); // lambda → (pointer →) function call
...
auto const sortDouble = SORTFUNC(double); // lambda value → lambda object
double* (*sF)(size_t nmemb, double[nmemb]) = sortDouble; // conversion
...
double* ap = sortDouble(4, (double[]){ 5, 8.9, 0.1, 99, });
double B[27] = { /* some values ... */ };
sF(27, B); // reuses the same function
...
double* (*sG)(size_t nmemb, double[nmemb]) = SORTFUNC(double); // conversion

```

This code evaluates the macro `SORTFUNC` twice, therefore in total four lambda expressions are formed.

The function literals of the “comparison lambdas” are not operands of a function call expression, and so by conversion a pointer to function is formed and passed to the corresponding call of `qsort`. Since the respective captures are empty, the effect is as if to define two comparison functions, that could equally well be implemented as **static** functions with auxiliary names and these names could be used to pass the function pointers to `qsort`.

The outer lambdas are again without capture. In the first case, for `long`, the lambda value is subject to a function call, and it is unspecified if the function call uses a specific lambda type or directly uses a function pointer. For the second, a copy of the lambda value is stored in the variable `sortDouble` and then converted to a function pointer `sF`. Other than for the difference in the function arguments, the effect of calling the lambda value (for the compound literal) or the function pointer (for array `B`) is the same.

For optimization purposes, an implementation may fold lambda values that are expanded at different points of the program such that effectively only one function is generated. For example here the function pointers `sF` and `sG` may or may not be equal.

### 15 EXAMPLE 3

```

void matmult(size_t k, size_t l, size_t m,
             double const A[k][l], double const B[l][m], double const C[k][m]) {
    // dot product with stride of m for B
    // ensure constant propagation of l and m
    auto const λδ = [l, m](double const v[l], double const B[l][m], size_t m0) {
        double ret = 0.0;
        for (size_t i = 0; i < l; ++i) {
            ret += v[i]*B[i][m0];
        }
        return ret;
    };
    // vector matrix product
    // ensure constant propagation of l and m, and accessibility of λδ
    auto const λμ = [l, m, λδ](double const v[l], double const B[l][m], double res[m]) {
        for (size_t m0 = 0; m0 < m; ++m0) {
            res[m0] = λδ(v, B, m0);
        }
    };
    for (size_t k0 = 0; k0 < k; ++k0) {
        double const (*Ap)[l] = A[k0];
        double (*Cp)[m] = C[k0];
        λμ(*Ap, B, *Cp);
    }
}

```

This function evaluates two closures;  $\lambda\delta$  has a return type of **double**,  $\lambda\mu$  of **void**. Both lambda values serve repeatedly as first operand to function evaluation but the evaluation of the captures is only done once for each of the closures. For the purpose of optimization, an implementation could generate copies of the underlying functions for each evaluation of such a closure such that the values of the captures `l` and `m` are replaced on a machine instruction level.

```
sizeof array / sizeof array[0]
```

- 8 **EXAMPLE 3** In this example, the size of a variable length array is computed and returned from a function:

```
#include <stddef.h>

size_t fsize3(int n)
{
    char b[n+3];      // variable length array
    return sizeof b;   // execution time sizeof
}

int main()
{
    size_t size;
    size = fsize3(10); // fsize3 returns 13
    return 0;
}
```

**Forward references:** common definitions `<stddef.h>` (7.19), declarations (6.7), structure and union specifiers (6.7.2.1), type names (6.7.7), array declarators (6.7.6.2).

## 6.5.4 Cast operators

### Syntax

- 1 *cast-expression:*
- ```
unary-expression
( type-name ) cast-expression
```

### Constraints

- 2 Unless the type name specifies a void type, the type name shall specify atomic, qualified, or unqualified scalar type, and the operand shall have scalar type, or, the type name shall specify an atomic, qualified, or unqualified pointer to function with prototype, and the operand is a function literal such a conversion (6.3.2.1) from the function literal to the function pointer type is defined.
- 3 Conversions that involve pointers, other than where permitted by the constraints of 6.5.16.1, shall be specified by means of an explicit cast.
- 4 A pointer type shall not be converted to any floating type. A floating type shall not be converted to any pointer type.

### Semantics

- 5 Preceding an expression by a parenthesized type name converts the value of the expression to the unqualified version of the named type. This construction is called a *cast*.<sup>116)</sup> A cast that specifies no conversion has no effect on the type or value of an expression.
- 6 If the value of the expression is represented with greater range or precision than required by the type named by the cast (6.3.1.8), then the cast specifies a conversion even if the type of the expression is the same as the named type and removes any extra range and precision.

**Forward references:** equality operators (6.5.9), function declarators (including prototypes) (6.7.6.3), simple assignment (6.5.16.1), type names (6.7.7).

## 6.5.5 Multiplicative operators

### Syntax

- 1 *multiplicative-expression:*
- ```
cast-expression
```

<sup>116)</sup>A cast does not yield an lvalue.

- the left operand has atomic, qualified, or unqualified pointer type, and (considering the type the left operand would have after lvalue conversion) one operand is a pointer to an object type, and the other is a pointer to a qualified or unqualified version of **void**, and the type pointed to by the left has all the qualifiers of the type pointed to by the right;
- the left operand is an atomic, qualified, or unqualified pointer to function with a prototype, the right operand is a function literal, and the prototypes of the function pointer and of the function literal shall be such that a conversion from the function literal to the function pointer type is defined;<sup>125)</sup>
- the left operand is an atomic, qualified, or unqualified pointer, and the right is a null pointer constant; or
- the left operand has type atomic, qualified, or unqualified **\_Bool**, and the right is a pointer.

### Semantics

- 2 In *simple assignment* (**=**), the value of the right operand is converted to the type of the assignment expression and replaces the value stored in the object designated by the left operand.
- 3 If the value being stored in an object is read from another object that overlaps in any way the storage of the first object, then the overlap shall be exact and the two objects shall have qualified or unqualified versions of a compatible type; otherwise, the behavior is undefined.
- 4 **EXAMPLE 1** In the program fragment

```
int f(void);
char c;
/* ... */
if ((c = f()) == -1)
    /* ... */
```

the **int** value returned by the function could be truncated when stored in the **char**, and then converted back to **int** width prior to the comparison. In an implementation in which “plain” **char** has the same range of values as **unsigned char** (and **char** is narrower than **int**), the result of the conversion cannot be negative, so the operands of the comparison can never compare equal. Therefore, for full portability, the variable **c** would be declared as **int**.

- 5 **EXAMPLE 2** In the fragment:

```
char c;
int i;
long l;

l = (c = i);
```

the value of **i** is converted to the type of the assignment expression **c = i**, that is, **char** type. The value of the expression enclosed in parentheses is then converted to the type of the outer assignment expression, that is, **long int** type.

- 6 **EXAMPLE 3** Consider the fragment:

```
const char **cpp;
char *p;
const char c = 'A';

cpp = &p;           // constraint violation
*cpp = &c;          // valid
*p = 0;            // valid
```

The first assignment is unsafe because it would allow the following valid code to attempt to change the value of the const object **c**.

<sup>125)</sup> Note that lambda types never appear as the left operand of an assignment. If they are the right operand, they are function literals and the left operand is a function pointer with a compatible prototype.



$\ast$  *type-qualifier-list*<sub>opt</sub> *pointer*  
*type-qualifier-list*:  
     *type-qualifier*  
     *type-qualifier-list* *type-qualifier*  
*parameter-type-list*:  
     *parameter-list*  
     *parameter-list* , ...  
*parameter-list*:  
     *parameter-declaration*  
     *parameter-list* , *parameter-declaration*  
*parameter-declaration*:  
     *declaration-specifiers* *declarator*  
     *declaration-specifiers* *abstract-declarator*<sub>opt</sub>  
*identifier-list*:  
     *identifier*  
     *identifier-list* , *identifier*

### Semantics

- 2 Each declarator declares one identifier, and asserts that when an operand of the same form as the declarator appears in an expression, it designates a function or object with the scope, storage duration, and type indicated by the declaration specifiers.
- 3 A *full declarator* is a declarator that is not part of another declarator. If, in the nested sequence of declarators in a full declarator, there is a declarator specifying a variable length array type, the type specified by the full declarator is said to be *variably modified*. Furthermore, any type derived by declarator type derivation from a variably modified type is itself variably modified.
- 4 In the following subclauses, consider a declaration

**T D1**

where **T** contains the declaration specifiers that specify a type *T* (such as **int**) and **D1** is a declarator that contains an identifier *ident*. The type specified for the identifier *ident* in the various forms of declarator is described inductively using this notation.

- 5 If, in the declaration "**T D1**", **D1** has the form

*identifier*

then the type specified for *ident* is *T*.

- 6 If, in the declaration "**T D1**", **D1** has the form

( **D** )

then *ident* has the type specified by the declaration "**T D**". Thus, a declarator in parentheses is identical to the unparenthesized declarator, but the binding of complicated declarators may be altered by parentheses.

### Implementation limits

- 7 As discussed in 5.2.4.1, an implementation may limit the number of pointer, array, and function declarators that modify an arithmetic, structure, union, or **void** type, either directly or via one or more **typedef** s.

**Forward references:** array declarators (6.7.6.2), type definitions (6.7.8)–, [type inference \(6.7.11\)](#).

#### 6.7.6.1 Pointer declarators

##### Semantics

- 1 If, in the declaration "**T D1**", **D1** has the form

$\ast$  *type-qualifier-list*<sub>opt</sub> **D**

and the type specified for *ident* in the declaration "**T D**" is "*derived-declarator-type-list T*", then the

declare a typedef name `t` with type **signed int**, a typedef name `plain` with type **int**, and a structure with three bit-field members, one named `t` that contains values in the range  $[0, 15]$ , an unnamed const-qualified bit-field which (if it could be accessed) would contain values in either the range  $[-15, +15]$  or  $[-16, +15]$ , and one named `r` that contains values in one of the ranges  $[0, 31]$ ,  $[-15, +15]$ , or  $[-16, +15]$ . (The choice of range is implementation-defined.) The first two bit-field declarations differ in that **unsigned** is a type specifier (which forces `t` to be the name of a structure member), while **const** is a type qualifier (which modifies `t` which is still visible as a typedef name). If these declarations are followed in an inner scope by

```
t f(t (t));
long t;
```

then a function `f` is declared with type “function returning **signed int** with one unnamed parameter with type pointer to function returning **signed int** with one unnamed parameter with type **signed int**”, and an identifier `t` with type **long int**.

- 7 **EXAMPLE 4** On the other hand, typedef names can be used to improve code readability. All three of the following declarations of the **signal** function specify exactly the same type, the first without making use of any typedef names.

```
typedef void fv(int), (*pfv)(int);

void (*signal(int, void (*)(int)))(int);
fv *signal(int, fv *);
pfv signal(int, pfv);
```

- 8 **EXAMPLE 5** If a typedef name denotes a variable length array type, the length of the array is fixed at the time the typedef name is defined, not each time it is used:

```
void copyt(int n)
{
    typedef int B[n]; // B is n ints, n evaluated now
    n += 1;
    B a; // a is n ints, n without += 1
    int b[n]; // a and b are different sizes
    for (int i = 1; i < n; i++)
        a[i-1] = b[i];
}
```

## 6.7.9 typeof specifier

### Syntax

- 1 *typeof-specifier*:
- ```
typeof ( type-name )
typeof ( expression )
```

### Constraints

- 2 The type name or expression shall be valid and have a function or object type, but not a lambda type. No new type declaration shall be formed by the type name or expression themselves.<sup>160)</sup>

### Semantics

- 3 A **typeof** specifier can be used in places where other type specifiers are used to declare or define objects, members or functions. It stands in for the unmodified type of the type name or expression, even where the expression cannot be used for type inference of its type (opaque types, function types, array types), where a type-qualification should not be dropped, or where an identifier may only be accessed for its type without evaluating it (within lambda expressions).
- 4 If it does not have a variably modified (VM) type, the type name or expression is not evaluated. For VM types, the same rules for evaluation as for **sizeof** expressions apply. Analogous to **typedef**, a

<sup>160)</sup>This could for example happen if the expression contained the forward declaration of a tag type, such as in `(struct newStruct*) 0` where `struct newStruct` has not yet been declared, or if it uses a compound literal that declares a new structure or union type in its *type-name* component.

```
{
    struct S l = { 1, .t = x, .t.l = 41, };
}
```

The value of `l.t.k` is 42, because implicit initialization does not override explicit initialization.

37 **EXAMPLE 13** Space can be “allocated” from both ends of an array by using a single designator:

```
int a[MAX] = {
    1, 3, 5, 7, 9, [MAX-5] = 8, 6, 4, 2, 0
};
```

38 In the above, if `MAX` is greater than ten, there will be some zero-valued elements in the middle; if it is less than ten, some of the values provided by the first five initializers will be overridden by the second five.

39 **EXAMPLE 14** Any member of a union can be initialized:

```
union { /* ... */ } u = {.any_member = 42 };
```

**Forward references:** common definitions `<stddef.h>` (7.19).

## 6.7.11 Type inference

### Constraints

- 1 An underspecified declaration shall contain the storage class specifier **auto**.
- 2 For an underspecified declaration of identifiers that is not a definition a prior definition for each identifier shall be visible and there shall be a **typeof** specifier *type* that if used to replace the **auto** specifier makes the adjusted declaration a valid declaration for each of the identifiers.
- 3 For an underspecified declaration that is also a definition of an object and that is not the declaration of a parameter, the init-declarator corresponding to the object shall be of one of the forms

```
declarator = assignment-expression
declarator = { assignment-expression }
declarator = { assignment-expression , }
```

such that the declarator does not declare an array. If the assignment expression has lambda type, the declaration shall only define one object and shall only consist of storage class specifiers, qualifiers, the identifier that is to be declared, and the initializer.

- 4 ~~For~~ Unless it is the definition of an object with an assignment expression of lambda type as above, for an underspecified declaration that is also a definition there shall be a **typeof** specifier *type* for a complete, non-atomic and unqualified type that if used to replace the **auto** specifier makes the adjusted declaration a valid declaration.<sup>166)</sup> If it is the definition of a function, it shall not additionally define objects and the return type of the function after adjustment shall be the same as determined from **return** statements (or the lack thereof) as in 6.9.1. Otherwise, *type* shall be such that for all defined objects the assignment expression in the init-declarator, after possible lvalue, array-to-pointer or function-to-pointer conversion, has the non-atomic, unqualified type of the declared object.<sup>167)</sup>
- 5 For the correspondence of the declared type of an object and the type of its initializer, integer types of the same rank and signedness but that are nevertheless different types shall not be considered.<sup>168)</sup> If the assignment-expression is the evaluation of a bit-field designator, the inferred type shall be the standard integer type that would be chosen by a generic primary expression with the that bit-field as controlling expression. If *type* is a VM type, the variable array bounds shall be such that the declared types for all defined objects and their assignment expression correspond as required for all possible

<sup>166)</sup>The qualification of the type of an lvalue that is the assignment expression, or the fact that it is atomic, can never be used to infer such a property of the type of the defined object.

<sup>167)</sup>For most assignment expressions of integer or floating point type, there are several types that would make such a declaration valid. The second part of the constraint ensures that among these a unique type is determined that does not need further conversion to be a valid initializer for the object.

<sup>168)</sup>This can for example be two different enumerated types that are compatible to the same basic type. Note nevertheless, that enumeration constants have type **int**, so using these will never lead to the inference of an enumerated type.

executions of the current function.

### Description

- 6 ~~Provided~~ Although there is no syntax derivation to form declarators of lambda type, values of lambda type can be used as assignment expression and the inferred type is that lambda type, possibly qualified. Otherwise, provided the constraints above are respected, in an underspecified declaration the type of the declared identifiers is the type after the declaration would have been adjusted by a choice for *type* as described. If the declaration is also an object definition, the assignment expressions that are used to determine types and initial values of the objects are evaluated at most once; the scope rules as described in 6.2.1 then also prohibit the use of the identifier of an object within the assignment expression that determines its type and initial value.
- 7 **NOTE 1** Because of the relatively complex syntax and semantics of type specifiers, the requirements for *type* use a **typeof** specifier. If for example the identifier or tag name of the type of the initializer expression *v* in the initializer of *x* is shadowed

```
auto x = v;
```

a type *type* as required can still be found and the definition can be adjusted as follows:

```
typeof(v) x = v;
```

Such a possible adjustment notwithstanding, if *v* is a VM type, the requirements ensure that *v* is evaluated at most once.

- 8 **NOTE 2** The scope of the identifier for which the type is inferred only starts after the end of the initializer (6.2.1), so the assignment expression cannot use the identifier to refer to the object or function that is declared, for example to take its address. Any use of the identifier in the initializer is invalid, even if an entity with the same name exists in an outer scope.

```
{
    double a = 7;
    double b = 9;
    {
        double b = b * b;    // error, RHS uses uninitialized variable
        printf("%g\n", a);  // valid, uses "a" from outer scope, prints 7
        auto a = a * a;     // error, "a" from outer scope is already shadowed
    }
    {
        auto b = a * a;      // valid, uses "a" from outer scope
        auto a = b;          // valid, shadows "a" from outer scope
        ...
        printf("%g\n", a);  // valid, uses "a" from inner scope, prints 49
    }
    ...
}
```

- 9 **NOTE 3** Declarations that are the definition of several objects, may make type inference difficult and not portable.

```
enum A { aVal, } aObj = aVal;
enum B { bVal, } bObj = bVal;
int au = aObj, bu = bObj;    // valid, values have type compatible to int
auto ax = aObj, bx = bObj;   // invalid, same rank but different types
auto ay = aObj;              // valid, ay has type enum A
auto by = bObj;              // valid, by has type enum B
auto az = aVal, bz = bVal;   // valid, az and bz have type int
struct set { int bits:32; } X = { .bits = 37, };
auto k = 37, m = X.bits;     // possibly valid or invalid
double aVM[r];
double bVM[s];
double cVM[3];
double dVM[r];
auto vmPa = &aVM, vmPa = &bVM; // invalid, different types for r != s
auto vmPa = &aVM, vmPc = &cVM; // invalid, even if for some executions r is 3
auto vmPa = &aVM, vmPd = &dVM; // valid, same array sizes in all executions
```

## 6.8 Statements and blocks

### Syntax

- 1 *statement*:
- labeled-statement*
  - compound-statement*
  - expression-statement*
  - selection-statement*
  - iteration-statement*
  - jump-statement*

### Semantics

- 2 A *statement* specifies an action to be performed. Except as indicated, statements are executed in sequence.
- 3 A *block* allows a set of declarations and statements to be grouped into one syntactic unit. The initializers of objects that have automatic storage duration, and the variable length array declarators of ordinary identifiers with block scope, are evaluated and the values are stored in the objects (including storing an indeterminate value in objects without an initializer) each time the declaration is reached in the order of execution, as if it were a statement, and within each declaration in the order that declarators appear.
- 4 A *full expression* is an expression that is not part of another expression, nor part of a declarator or abstract declarator. There is also an implicit full expression in which the non-constant size expressions for a variably modified type are evaluated; within that full expression, the evaluation of different size expressions are unsequenced with respect to one another. There is a sequence point between the evaluation of a full expression and the evaluation of the next full expression to be evaluated.
- 5 **NOTE** Each of the following is a full expression:
- a full declarator for a variably modified type,
  - an initializer that is not part of a compound literal,
  - the expression in an expression statement,
  - the controlling expression of a selection statement (**if** or **switch**),
  - the controlling expression of a **while** or **do** statement,
  - each of the (optional) expressions of a **for** statement,
  - the (optional) expression in a **return** statement.

While a constant expression satisfies the definition of a full expression, evaluating it does not depend on nor produce any side effects, so the sequencing implications of being a full expression are not relevant to a constant expression.

**Forward references:** expression and null statements (6.8.3), selection statements (6.8.4), iteration statements (6.8.5), the **return** statement (6.8.6.4).

### 6.8.1 Labeled statements

#### Syntax

- 1 *labeled-statement*:
- identifier* : *statement*
  - case** *constant-expression* : *statement*
  - default** : *statement*

#### Constraints

- 2 A **case** or **default** label shall appear only in a **switch** statement ~~that is associated with the same function body as the statement to which the label is attached.~~<sup>169)</sup> Further constraints on such labels are discussed under the **switch** statement.

<sup>169)</sup> Thus, a label that appears within a lambda expression may only be associated to a switch statement within the body of the lambda.

### 6.8.5.3 The for statement

- 1 The statement

```
for (clause-1; expression-2; expression-3) statement
```

behaves as follows: The expression *expression-2* is the controlling expression that is evaluated before each execution of the loop body. The expression *expression-3* is evaluated as a void expression after each execution of the loop body. If *clause-1* is a declaration, the scope of any identifiers it declares is the remainder of the declaration and the entire loop, including the other two expressions; it is reached in the order of execution before the first evaluation of the controlling expression. If *clause-1* is an expression, it is evaluated as a void expression before the first evaluation of the controlling expression.<sup>175)</sup>

- 2 Both *clause-1* and *expression-3* can be omitted. An omitted *expression-2* is replaced by a nonzero constant.

### 6.8.6 Jump statements

#### Syntax

- 1 *jump-statement*:
 

```
goto identifier ;
continue ;
break ;
return expressionopt ;
```

#### Constraints

- 2 No jump statement other than **return** shall have a target that is found in another function body.<sup>176)</sup>

#### Semantics

- 3 A jump statement causes an unconditional jump to another place.

#### 6.8.6.1 The goto statement

##### Constraints

- 1 The identifier in a **goto** statement shall name a label located somewhere in the enclosing function body. A **goto** statement shall not jump from outside the scope of an identifier having a variably modified type to inside the scope of that identifier.<sup>177)</sup>

##### Semantics

- 2 A **goto** statement causes an unconditional jump to the statement prefixed by the named label in the enclosing function.
- 3 **EXAMPLE 1** It is sometimes convenient to jump into the middle of a complicated set of statements. The following outline presents one possible approach to a problem based on these three assumptions:
  1. The general initialization code accesses objects only visible to the current function.
  2. The general initialization code is too large to warrant duplication.
  3. The code to determine the next operation is at the head of the loop. (To allow it to be reached by **continue** statements, for example.)

```
/* ... */
goto first_time;
for (;;) {
```

<sup>175)</sup>Thus, *clause-1* specifies initialization for the loop, possibly declaring one or more variables for use in the loop; the controlling expression, *expression-2*, specifies an evaluation made before each iteration, such that execution of the loop continues until the expression compares equal to 0; and *expression-3* specifies an operation (such as incrementing) that is performed after each iteration.

<sup>176)</sup>Thus jump statements other than **return** may not jump between different functions or cross the boundaries of a lambda expression, that is, they may not jump into or out of the function body of a lambda. Other features such as signals (7.14) and long jumps (7.13) may delegate control to points of the program that do not fall under these constraints.

<sup>177)</sup>The visibility of labels is restricted such that a **goto** statement that jumps into or out of a different function body, even if it is nested within a lambda, is a constraint violation.

```

    // determine next operation
    /* ... */
    if (need to reinitialize) {
        // reinitialize-only code
        /* ... */
    first_time:
        // general initialization code
        /* ... */
        continue;
    }
    // handle other operations
    /* ... */
}

```

- 4 **EXAMPLE 2** A **goto** statement is not allowed to jump past any declarations of objects with variably modified types. A jump within the scope, however, is permitted.

```

goto lab3;           // invalid: going INTO scope of VLA.
{
    double a[n];
    a[j] = 4.4;
lab3:
    a[j] = 3.3;
    goto lab4;       // valid: going WITHIN scope of VLA.
    a[j] = 5.5;
lab4:
    a[j] = 6.6;
}
goto lab4;           // invalid: going INTO scope of VLA.

```

### 6.8.6.2 The **continue** statement

#### Constraints

- 1 A **continue** statement shall appear only in or as a loop body that is associated to the same function body.<sup>178)</sup>

#### Semantics

- 2 A **continue** statement causes a jump to the loop-continuation portion of the smallest enclosing iteration statement; that is, to the end of the loop body. More precisely, in each of the statements

```

while (/* ... */) {
    /* ... */
    continue;
    /* ... */
contin:;
}

```

```

do {
    /* ... */
    continue;
    /* ... */
contin:;
} while (/* ... */);

```

```

for (/* ... */) {
    /* ... */
    continue;
    /* ... */
contin:;
}

```

unless the **continue** statement shown is in an enclosed iteration statement (in which case it is interpreted within that statement), it is equivalent to **goto** contin;<sup>179)</sup>

### 6.8.6.3 The **break** statement

#### Constraints

- 1 A **break** statement shall appear only in or as a switch body or loop body that is associated to the same function body.<sup>180)</sup>

<sup>178)</sup> Thus a **continue** statement by itself may not be used to terminate the execution of the body of a lambda expression.

<sup>179)</sup> Following the `contin:` label is a null statement.

<sup>180)</sup> Thus a **break** statement by itself may not be used to terminate the execution of the body of a lambda expression.



## Semantics

- 2 A **break** statement terminates execution of the smallest enclosing **switch** or iteration statement.

### 6.8.6.4 The return statement

#### Constraints

- 1 A **return** statement with an expression shall not appear in a function body whose return type is **void**. A **return** statement without an expression shall only appear in a function body whose return type is **void**.
- 2 For a function ~~that has~~ body that corresponds to an underspecified definition of a function or to a lambda, all **return** statements shall provide expressions with a consistent type or none at all. That is, if any **return** statement has an expression, all **return** statements shall have an expression (after lvalue, array-to-pointer or function-to-pointer conversion) with the same type; otherwise all **return** expressions shall have no expression.

#### Semantics

- 3 A **return** statement is associated to the innermost function body in which appears. It evaluates the expression, if any, terminates the execution of ~~the that~~ function body and returns control to ~~the caller~~ its caller; if it has an expression, the value of the expression is returned to the caller as the value of the function call expression. A function body may have any number of **return** statements.
- 4 If a **return** statement with an expression is executed, the value of the expression is returned to the caller as the value of the function call expression. If the expression has a type different from the return type of the function in which it appears, the value is converted as if by assignment to an object having the return type of the function.<sup>181)</sup>
- 5 For a lambda or for a function that has an underspecified definition, the return type is determined by the lexically first **return** statement, if any, that is associated to the function body and is specified as the type of that expression, if any, after lvalue, array-to-pointer, function-to-pointer conversion, or as **void** if there is no expression.
- 6 **EXAMPLE** In:

```
struct s { double i; } f(void);
union {
    struct {
        int f1;
        struct s f2;
    } u1;
    struct {
        struct s f3;
        int f4;
    } u2;
} g;

struct s f(void)
{
    return g.u1.f2;
}

/* ... */
g.u2.f3 = f();
```

there is no undefined behavior, although there would be if the assignment were done directly (without using a function call to fetch the value).

<sup>181)</sup>The **return** statement is not an assignment. The overlap restriction of 6.5.16.1 does not apply to the case of function return. The representation of floating-point values can have wider range or precision than implied by the type; a cast can be used to remove this extra range and precision.

- If an argument to a function has an invalid value (such as a value outside the domain of the function, or a pointer outside the address space of the program, or a null pointer, or a pointer to non-modifiable storage when the corresponding parameter is not const-qualified) or a type (after default argument promotion) not expected by a function with a variable number of arguments, the behavior is undefined.
  - If a function argument is described as being an array, the pointer actually passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are in fact valid.
  - Any function declared in a header may be additionally implemented as a function-like macro defined in the header, so if a library function is declared explicitly when its header is included, one of the techniques shown below can be used to ensure the declaration is not affected by such a macro. Any macro definition of a function can be suppressed locally by enclosing the name of the function in parentheses, because the name is then not followed by the left parenthesis that indicates expansion of a macro function name. For the same syntactic reason, it is permitted to take the address of a library function even if it is also defined as a macro.<sup>210)</sup> The use of **#undef** to remove any macro definition will also ensure that an actual function is referred to.
  - Any invocation of a library function that is implemented as a macro shall expand to code that evaluates each of its arguments exactly once, fully protected by parentheses where necessary, so it is generally safe to use arbitrary expressions as arguments.<sup>211)</sup>
  - Likewise, those function-like macros described in the following subclauses may be invoked in an expression anywhere a function with a compatible return type could be called.<sup>212)</sup>
  - All object-like macros listed as expanding to integer constant expressions shall additionally be suitable for use in **#if** preprocessing directives.
- 2 Provided that a library function can be declared without reference to any type defined in a header, it is also permissible to declare the function and use it without including its associated header.
  - 3 There is a sequence point immediately before a library function returns.
  - 4 The functions in the standard library are not guaranteed to be reentrant and may modify objects with static or thread storage duration.<sup>213)</sup>
  - 5 Unless explicitly stated otherwise in the detailed descriptions that follow, library functions shall prevent data races as follows: A library function shall not directly or indirectly access objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's arguments. A library function shall not directly or indirectly modify objects accessible by threads other than the current thread unless the objects are accessed directly

<sup>210)</sup>This means that an implementation is required to provide an actual function for each library function, even if it also provides a macro for that function.

<sup>211)</sup>Such macros might not contain the sequence points that the corresponding function calls do. [Nevertheless, it is recommended that implementations provide the same sequencing properties as for a function call, by, for example, wrapping the macro expansion in a suitable lambda expression.](#)

<sup>212)</sup>Because external identifiers and some macro names beginning with an underscore are reserved, implementations can provide special semantics for such names. For example, the identifier `_BUILTIN_abs` could be used to indicate generation of in-line code for the `abs` function. Thus, the appropriate header could specify

```
#define abs(x) _BUILTIN_abs(x)
```

for a compiler whose code generator will accept it.

In this manner, a user desiring to guarantee that a given library function such as `abs` will be a genuine function can write

```
#undef abs
```

whether the implementation's header provides a macro implementation of `abs` or a built-in implementation. The prototype for the function, which precedes and is hidden by any macro definition, is thereby revealed also.

<sup>213)</sup>Thus, a signal handler cannot, in general, call standard library functions.

## Description

- 2 The **longjmp** function restores the environment saved by the most recent invocation of the **setjmp** macro in the same invocation of the program with the corresponding **jmp\_buf** argument. If there has been no such invocation, or if the invocation was from another thread of execution, or if the function [or lambda](#) containing the invocation of the **setjmp** macro has terminated execution<sup>273)</sup> in the interim, or if the invocation of the **setjmp** macro was within the scope of an identifier with variably modified type and execution has left that scope in the interim, the behavior is undefined.
- 3 All accessible objects have values, and all other components of the abstract machine<sup>274)</sup> have state, as of the time the **longjmp** function was called, except that the values of objects of automatic storage duration that are local to the function [or lambda](#) containing the invocation of the corresponding **setjmp** macro that do not have volatile-qualified type and have been changed between the **setjmp** invocation and **longjmp** call are indeterminate.

## Returns

- 4 After **longjmp** is completed, thread execution continues as if the corresponding invocation of the **setjmp** macro had just returned the value specified by **val**. The **longjmp** function cannot cause the **setjmp** macro to return the value 0; if **val** is 0, the **setjmp** macro returns the value 1.
- 5 **EXAMPLE** The **longjmp** function that returns control back to the point of the **setjmp** invocation might cause memory associated with a variable length array object to be squandered.

```
#include <setjmp.h>
jmp_buf buf;
void g(int n);
void h(int n);
int n = 6;

void f(void)
{
    int x[n];           // valid: f is not terminated
    setjmp(buf);
    g(n);
}

void g(int n)
{
    int a[n];           // a may remain allocated
    h(n);
}

void h(int n)
{
    int b[n];           // b may remain allocated
    longjmp(buf, 2);    // might cause memory loss
}
```

<sup>273)</sup>For example, by executing a **return** statement or because another **longjmp** call has caused a transfer to a **setjmp** invocation in a function [or lambda](#) earlier in the set of nested calls.

<sup>274)</sup>This includes, but is not limited to, the floating-point status flags and the state of open files.

## 7.14 Signal handling <signal.h>

- 1 The header <signal.h> declares a type and two functions and defines several macros, for handling various *signals* (conditions that may be reported during program execution).
- 2 The type defined is

```
sig_atomic_t
```

which is the (possibly volatile-qualified) integer type of an object that can be accessed as an atomic entity, even in the presence of asynchronous interrupts.

- 3 The macros defined are

```
SIG_DFL
SIG_ERR
SIG_IGN
```

which expand to constant expressions with distinct values that have type compatible with the second argument to, and the return value of, the **signal** function, and whose values compare unequal to the address of any declarable function; and the following, which expand to positive integer constant expressions with type **int** and distinct values that are the signal numbers, each corresponding to the specified condition:

**SIGABRT** abnormal termination, such as is initiated by the **abort** function

**SIGFPE** an erroneous arithmetic operation, such as zero divide or an operation resulting in overflow

**SIGILL** detection of an invalid function image, such as an invalid instruction

**SIGINT** receipt of an interactive attention signal

**SIGSEGV** an invalid access to storage

**SIGTERM** a termination request sent to the program

- 4 An implementation need not generate any of these signals, except as a result of explicit calls to the **raise** function. Additional signals and pointers to undeclarable functions, with macro definitions beginning, respectively, with the letters **SIG** and an uppercase letter or with **SIG\_** and an uppercase letter,<sup>275)</sup> may also be specified by the implementation. The complete set of signals, their semantics, and their default handling is implementation-defined; all signal numbers shall be positive.

### 7.14.1 Specify signal handling

#### 7.14.1.1 The **signal** function

##### Synopsis

- 1 

```
#include <signal.h>
void (*signal(int sig, void (*func)(int)))(int);
```

##### Description

- 2 The **signal** function chooses one of three ways in which receipt of the signal number **sig** is to be subsequently handled. If the value of **func** is **SIG\_DFL**, default handling for that signal will occur. If the value of **func** is **SIG\_IGN**, the signal will be ignored. Otherwise, **func** shall point to a function or shall be the result of a conversion of a function literal to a function pointer. The function or lambda value is then to be called when that signal occurs<sup>276)</sup>. An invocation of such a function or function literal because of a signal, or (recursively) of any further functions or lambdas called by that invocation (other than functions in the standard library),<sup>276)</sup> is called a *signal handler*.

<sup>275)</sup>See “future library directions” (7.31.7). The names of the signal numbers reflect the following terms (respectively): abort, floating-point exception, illegal instruction, interrupt, segmentation violation, and termination.

<sup>276)</sup>This includes functions called indirectly via standard library functions (e.g., a **SIGABRT** handler called via the **abort** function).

- 3 When a signal occurs and `func` points to a function,<sup>277)</sup> it is implementation-defined whether the equivalent of `signal(sig, SIG_DFL)`; is executed or the implementation prevents some implementation-defined set of signals (at least including `sig`) from occurring until the current signal handling has completed; in the case of `SIGILL`, the implementation may alternatively define that no action is taken. Then the equivalent of `(*func)(sig)`; is executed. If and when the function returns, if the value of `sig` is `SIGFPE`, `SIGILL`, `SIGSEGV`, or any other implementation-defined value corresponding to a computational exception, the behavior is undefined; otherwise the program will resume execution at the point it was interrupted.
- 4 If the signal occurs as the result of calling the `abort` or `raise` function, the signal handler shall not call the `raise` function.
- 5 If the signal occurs other than as the result of calling the `abort` or `raise` function, the behavior is undefined if the signal handler refers to any object with static or thread storage duration that is not a lock-free atomic object other than by assigning a value to an object declared as `volatile sig_atomic_t`, or the signal handler calls any function in the standard library other than
  - the `abort` function,
  - the `_Exit` function,
  - the `quick_exit` function,
  - the functions in `<stdatomic.h>` (except where explicitly stated otherwise) when the atomic arguments are lock-free,
  - the `atomic_is_lock_free` function with any atomic argument, or
  - the `signal` function with the first argument equal to the signal number corresponding to the signal that caused the invocation of the handler. Furthermore, if such a call to the `signal` function results in a `SIG_ERR` return, the value of `errno` is indeterminate.<sup>278)</sup>
- 6 At program startup, the equivalent of

```
signal(sig, SIG_IGN);
```

may be executed for some signals selected in an implementation-defined manner; the equivalent of

```
signal(sig, SIG_DFL);
```

is executed for all other signals defined by the implementation.

- 7 Use of this function in a multi-threaded program results in undefined behavior. The implementation shall behave as if no library function calls the `signal` function.

### Returns

- 8 If the request can be honored, the `signal` function returns the value of `func` for the most recent successful call to `signal` for the specified signal `sig`. Otherwise, a value of `SIG_ERR` is returned and a positive value is stored in `errno`.

**Forward references:** the `abort` function (7.22.4.1), the `exit` function (7.22.4.4), the `_Exit` function (7.22.4.5), the `quick_exit` function (7.22.4.7).

## 7.14.2 Send signal

### 7.14.2.1 The `raise` function

#### Synopsis

- 1
 

```
#include <signal.h>
int raise(int sig);
```

<sup>277)</sup>Or, equivalently, it is the result of a conversion of a function literal to a function pointer.

<sup>278)</sup>If any signal is generated by an asynchronous signal handler, the behavior is undefined.

## 7.16 Variable arguments <stdarg.h>

- 1 The header <stdarg.h> declares a type and defines four macros, for advancing through a list of arguments whose number and types are not known to the called function when it is translated.
- 2 A function may be called with a variable number of arguments of varying types. As described in 6.9.1, its parameter list contains one or more parameters. The rightmost parameter plays a special role in the access mechanism, and will be designated *parmN* in this description.
- 3 The type declared is

**va\_list**

which is a complete object type suitable for holding information needed by the macros **va\_start**, **va\_arg**, **va\_end**, and **va\_copy**. If access to the varying arguments is desired, the called function shall declare an object (generally referred to as **ap** in this subclause) having type **va\_list**. The object **ap** may be passed as an argument to another function ~~;*if that function call; if the called function or lambda*~~ invokes the **va\_arg** macro with parameter **ap**, the value of **ap** in the calling function ~~or lambda~~ is indeterminate and shall be passed to the **va\_end** macro prior to any further reference to **ap**.<sup>279)</sup>

- 4 **NOTE** Because the . . . parameter syntax is not valid for lambda expressions, these macros can never be applied directly to process a variable list of arguments to the call of a lambda. In contrast to that, the type **va\_list** itself can be a parameter type of a lambda expression to process the argument list of a function.

### 7.16.1 Variable argument list access macros

- 1 The **va\_start** and **va\_arg** macros described in this subclause shall be implemented as macros, not functions. It is unspecified whether **va\_copy** and **va\_end** are macros or identifiers declared with external linkage. If a macro definition is suppressed in order to access an actual function, or a program defines an external identifier with the same name, the behavior is undefined. Each invocation of the **va\_start** and **va\_copy** macros shall be matched by a corresponding invocation of the **va\_end** macro in the same function ~~or lambda expression~~.

#### 7.16.1.1 The **va\_arg** macro

##### Synopsis

- 1

```
#include <stdarg.h>
type va_arg(va_list ap, type);
```

##### Description

- 2 The **va\_arg** macro expands to an expression that has the specified type and the value of the next argument in the call. The parameter **ap** shall have been initialized by the **va\_start** or **va\_copy** macro (without an intervening invocation of the **va\_end** macro for the same **ap**). Each invocation of the **va\_arg** macro modifies **ap** so that the values of successive arguments are returned in turn. The parameter *type* shall be a type name specified such that the type of a pointer to an object that has the specified type can be obtained simply by postfixing a \* to *type*. If there is no actual next argument, or if *type* is not compatible with the type of the actual next argument (as promoted according to the default argument promotions), the behavior is undefined, except for the following cases:

- one type is a signed integer type, the other type is the corresponding unsigned integer type, and the value is representable in both types;
- one type is pointer to **void** and the other is a pointer to a character type.

##### Returns

- 3 The first invocation of the **va\_arg** macro after that of the **va\_start** macro returns the value of the argument after that specified by *parmN*. Successive invocations return the values of the remaining arguments in succession.

<sup>279)</sup>It is permitted to create a pointer to a **va\_list** and pass that pointer to another function ~~or lambda~~, in which case the ~~original calling~~ function ~~or lambda~~ can make further use of the original list after the other function returns.



### 7.16.1.2 The **va\_copy** macro

#### Synopsis

```
1  #include <stdarg.h>
    void va_copy(va_list dest, va_list src);
```

#### Description

- 2 The **va\_copy** macro initializes **dest** as a copy of **src**, as if the **va\_start** macro had been applied to **dest** followed by the same sequence of uses of the **va\_arg** macro as had previously been used to reach the present state of **src**. Neither the **va\_copy** nor **va\_start** macro shall be invoked to reinitialize **dest** without an intervening invocation of the **va\_end** macro for the same **dest**.

#### Returns

- 3 The **va\_copy** macro returns no value.

### 7.16.1.3 The **va\_end** macro

#### Synopsis

```
1  #include <stdarg.h>
    void va_end(va_list ap);
```

#### Description

- 2 The **va\_end** macro facilitates a normal return from the function whose variable argument list was referred to by the expansion of the **va\_start** macro, or the function or lambda expression containing the expansion of the **va\_copy** macro, that initialized the **va\_list ap**. The **va\_end** macro may modify **ap** so that it is no longer usable (without being reinitialized by the **va\_start** or **va\_copy** macro). If there is no corresponding invocation of the **va\_start** or **va\_copy** macro, or if the **va\_end** macro is not invoked before the return, the behavior is undefined.

#### Returns

- 3 The **va\_end** macro returns no value.

### 7.16.1.4 The **va\_start** macro

#### Synopsis

```
1  #include <stdarg.h>
    void va_start(va_list ap, parmN);
```

#### Description

- 2 The **va\_start** macro shall be invoked before any access to the unnamed arguments.
- 3 The **va\_start** macro initializes **ap** for subsequent use by the **va\_arg** and **va\_end** macros. Neither the **va\_start** nor **va\_copy** macro shall be invoked to reinitialize **ap** without an intervening invocation of the **va\_end** macro for the same **ap**.
- 4 The parameter *parmN* is the identifier of the rightmost parameter in the variable parameter list in the function definition (the one just before the `, ...`). If the parameter *parmN* is declared with the **register** storage class, with a function or array type, or with a type that is not compatible with the type that results after application of the default argument promotions, the behavior is undefined.

#### Returns

- 5 The **va\_start** macro returns no value.
- 6 **EXAMPLE 1** The function **f1** gathers into an array a list of arguments that are pointers to strings (but not more than **MAXARGS** arguments), then passes the array as a single argument to function **f2**. The number of pointers is specified by the first argument to **f1**.



**Returns**

- 4 The **realloc** function returns a pointer to the new object (which may have the same value as a pointer to the old object), or a null pointer if the new object has not been allocated.

**7.22.4 Communication with the environment****7.22.4.1 The **abort** function****Synopsis**

```
1  #include <stdlib.h>
    _Noreturn void abort(void);
```

**Description**

- 2 The **abort** function causes abnormal program termination to occur, unless the signal **SIGABRT** is being caught and the signal handler does not return. Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined. An implementation-defined form of the status *unsuccessful termination* is returned to the host environment by means of the function call **raise(SIGABRT)**.

**Returns**

- 3 The **abort** function does not return to its caller.

**7.22.4.2 The **atexit** function****Synopsis**

```
1  #include <stdlib.h>
    int atexit(void (*func)(void));
```

**Description**

- 2 The **atexit** function registers the function or function literal pointed to by **func**, to be called without arguments at normal program termination.<sup>324</sup> It is unspecified whether a call to the **atexit** function that does not happen before the **exit** function is called will succeed.

**Environmental limits**

- 3 The implementation shall support the registration of at least 32 ~~functions~~function pointers.

**Returns**

- 4 The **atexit** function returns zero if the registration succeeds, nonzero if it fails.

**Forward references:** the **at\_quick\_exit** function (7.22.4.3), the **exit** function (7.22.4.4).

**7.22.4.3 The **at\_quick\_exit** function****Synopsis**

```
1  #include <stdlib.h>
    int at_quick_exit(void (*func)(void));
```

**Description**

- 2 The **at\_quick\_exit** function registers the function or function literal pointed to by **func**, to be called without arguments should **quick\_exit** be called.<sup>325</sup> It is unspecified whether a call to the **at\_quick\_exit** function that does not happen before the **quick\_exit** function is called will succeed.

<sup>324</sup>The **atexit** function registrations are distinct from the **at\_quick\_exit** registrations, so applications might need to call both registration functions with the same argument.

<sup>325</sup>The **at\_quick\_exit** function registrations are distinct from the **atexit** registrations, so applications might need to call both registration functions with the same argument.

**Environmental limits**

- 3 The implementation shall support the registration of at least 32 ~~functions~~function pointers.

**Returns**

- 4 The **at\_quick\_exit** function returns zero if the registration succeeds, nonzero if it fails.

**Forward references:** the **quick\_exit** function (7.22.4.7).

**7.22.4.4 The **exit** function****Synopsis**

```
1  #include <stdlib.h>
    _Noreturn void exit(int status);
```

**Description**

- 2 The **exit** function causes normal program termination to occur. No ~~functions~~function pointers registered by the **at\_quick\_exit** function are called. If a program calls the **exit** function more than once, or calls the **quick\_exit** function in addition to the **exit** function, the behavior is undefined.
- 3 First, all ~~functions~~function pointers registered by the **atexit** function are called, in the reverse order of their registration,<sup>326)</sup> except that a function pointer is called after any previously registered ~~functions~~function pointers that had already been called at the time it was registered. If, during the call to any such function or function literal, a call to the **longjmp** function is made that would terminate the call to the registered function or function literal, the behavior is undefined.
- 4 Next, all open streams with unwritten buffered data are flushed, all open streams are closed, and all files created by the **tmpfile** function are removed.
- 5 Finally, control is returned to the host environment. If the value of **status** is zero or **EXIT\_SUCCESS**, an implementation-defined form of the status *successful termination* is returned. If the value of **status** is **EXIT\_FAILURE**, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.

**Returns**

- 6 The **exit** function cannot return to its caller.

**7.22.4.5 The **\_Exit** function****Synopsis**

```
1  #include <stdlib.h>
    _Noreturn void _Exit(int status);
```

**Description**

- 2 The **\_Exit** function causes normal program termination to occur and control to be returned to the host environment. No ~~functions~~function pointers registered by the **atexit** function, the **at\_quick\_exit** function, or signal handlers registered by the **signal** function are called. The status returned to the host environment is determined in the same way as for the **exit** function (7.22.4.4). Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined.

**Returns**

- 3 The **\_Exit** function cannot return to its caller.

**7.22.4.6 The **getenv** function****Synopsis**

```
1  #include <stdlib.h>
```

<sup>326)</sup>Each function is called as many times as it was registered, and in the correct order with respect to other registered ~~functions~~function pointers.

```
char *getenv(const char *name);
```

### Description

- 2 The **getenv** function searches an *environment list*, provided by the host environment, for a string that matches the string pointed to by **name**. The set of environment names and the method for altering the environment list are implementation-defined. The **getenv** function need not avoid data races with other threads of execution that modify the environment list.<sup>327)</sup>
- 3 The implementation shall behave as if no library function calls the **getenv** function.

### Returns

- 4 The **getenv** function returns a pointer to a string associated with the matched list member. The string pointed to shall not be modified by the program, but may be overwritten by a subsequent call to the **getenv** function. If the specified **name** cannot be found, a null pointer is returned.

## 7.22.4.7 The **quick\_exit** function

### Synopsis

```
1 #include <stdlib.h>
   _Noreturn void quick_exit(int status);
```

### Description

- 2 The **quick\_exit** function causes normal program termination to occur. No ~~functions~~function pointers registered by the **atexit** function or signal handlers registered by the **signal** function are called. If a program calls the **quick\_exit** function more than once, or calls the **exit** function in addition to the **quick\_exit** function, the behavior is undefined. If a signal is raised while the **quick\_exit** function is executing, the behavior is undefined.
- 3 The **quick\_exit** function first calls all ~~functions~~function pointers registered by the **at\_quick\_exit** function, in the reverse order of their registration,<sup>328)</sup> except that a function pointer is called after any previously registered ~~functions~~function pointers that had already been called at the time it was registered. If, during the call to any such function or function literal, a call to the **longjmp** function is made that would terminate the call to the registered function pointer, the behavior is undefined.
- 4 Then control is returned to the host environment by means of the function call **\_Exit(status)**.

### Returns

- 5 The **quick\_exit** function cannot return to its caller.

## 7.22.4.8 The **system** function

### Synopsis

```
1 #include <stdlib.h>
   int system(const char *string);
```

### Description

- 2 If **string** is a null pointer, the **system** function determines whether the host environment has a *command processor*. If **string** is not a null pointer, the **system** function passes the string pointed to by **string** to that command processor to be executed in a manner which the implementation shall document; this might then cause the program calling **system** to behave in a non-conforming manner or to terminate.

### Returns

- 3 If the argument is a null pointer, the **system** function returns nonzero only if a command processor is available. If the argument is not a null pointer, and the **system** function does return, it returns an

<sup>327)</sup>Many implementations provide non-standard functions that modify the environment list.

<sup>328)</sup>Each function pointer is called as many times as it was registered, and in the correct order with respect to other registered ~~functions~~function pointers.

implementation-defined value.

## 7.22.5 Searching and sorting utilities

- 1 These utilities make use of a comparison function [or function literal](#) to search or sort arrays of unspecified type. Where an argument declared as `size_t nmemb` specifies the length of the array for a function, `nmemb` can have the value zero on a call to that function; the comparison function [or function literal](#) is not called, a search finds no matching element, and sorting performs no rearrangement. Pointer arguments on such a call shall still have valid values, as described in 7.1.4.
- 2 The implementation shall ensure that the second argument of the comparison function [or function literal](#) (when called from `bsearch`), or both arguments (when called from `qsort`), are pointers to elements of the array.<sup>329)</sup> The first argument when called from `bsearch` shall equal `key`.
- 3 The comparison function [or function literal](#) shall not alter the contents of the array. The implementation may reorder elements of the array between calls to the comparison function [or function literal](#), but shall not alter the contents of any individual element.
- 4 When the same objects (consisting of `size` bytes, irrespective of their current positions in the array) are passed more than once to the comparison function [or function literal](#), the results shall be consistent with one another. That is, for `qsort` they shall define a total ordering on the array, and for `bsearch` the same object shall always compare the same way with the key.
- 5 A sequence point occurs immediately before and immediately after each call to the comparison function [or function literal](#), and also between any call to the comparison function [or function literal](#) and any movement of the objects passed as arguments to that call.

### 7.22.5.1 The `bsearch` function

#### Synopsis

```
1  #include <stdlib.h>
   void *bsearch(const void *key, const void *base,
               size_t nmemb, size_t size,
               int (*compar)(const void *, const void *));
```

#### Description

- 2 The `bsearch` function searches an array of `nmemb` objects, the initial element of which is pointed to by `base`, for an element that matches the object pointed to by `key`. The size of each element of the array is specified by `size`.
- 3 The comparison function [or function literal](#) pointed to by `compar` is called with two arguments that point to the `key` object and to an array element, in that order. ~~The function~~ [A function call](#) shall return an integer less than, equal to, or greater than zero if the `key` object is considered, respectively, to be less than, to match, or to be greater than the array element. The array shall consist of: all the elements that compare less than, all the elements that compare equal to, and all the elements that compare greater than the `key` object, in that order.<sup>330)</sup>

#### Returns

- 4 The `bsearch` function returns a pointer to a matching element of the array, or a null pointer if no match is found. If two elements compare as equal, which element is matched is unspecified.

<sup>329)</sup>That is, if the value passed is `p`, then the following expressions are always nonzero:

```
((char *)p - (char *)base) % size == 0
(char *)p >= (char *)base
(char *)p < (char *)base + nmemb * size
```

<sup>330)</sup>In practice, the entire array is sorted according to the comparison function.

### 7.22.5.2 The **qsort** function

#### Synopsis

```
1  #include <stdlib.h>
    void qsort(void *base, size_t nmemb, size_t size,
               int (*compar)(const void *, const void *));
```

#### Description

- 2 The **qsort** function sorts an array of **nmemb** objects, the initial element of which is pointed to by **base**. The size of each object is specified by **size**.
- 3 The contents of the array are sorted into ascending order according to a comparison function or function literal pointed to by **compar**, which is called with two arguments that point to the objects being compared. ~~The function~~ A function call shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.
- 4 If two elements compare as equal, their order in the resulting sorted array is unspecified.

#### Returns

- 5 The **qsort** function returns no value.

## 7.22.6 Integer arithmetic functions

### 7.22.6.1 The **abs**, **labs** and **llabs** functions

#### Synopsis

```
1  #include <stdlib.h>
    int abs(int j);
    long int labs(long int j);
    long long int llabs(long long int j);
```

#### Description

- 2 The **abs**, **labs**, and **llabs** functions compute the absolute value of an integer **j**. If the result cannot be represented, the behavior is undefined.<sup>331)</sup>

#### Returns

- 3 The **abs**, **labs**, and **llabs**, functions return the absolute value.

### 7.22.6.2 The **div**, **ldiv**, and **lldiv** functions

#### Synopsis

```
1  #include <stdlib.h>
    div_t div(int numer, int denom);
    ldiv_t ldiv(long int numer, long int denom);
    lldiv_t lldiv(long long int numer, long long int denom);
```

#### Description

- 2 The **div**, **ldiv**, and **lldiv**, functions compute **numer/denom** and **numer%denom** in a single operation.

#### Returns

- 3 The **div**, **ldiv**, and **lldiv** functions return a structure of type **div\_t**, **ldiv\_t**, and **lldiv\_t**, respectively, comprising both the quotient and the remainder. The structures shall contain (in either order) the members **quot** (the quotient) and **rem** (the remainder), each of which has the same type as the arguments **numer** and **denom**. If either part of the result cannot be represented, the behavior is undefined.

<sup>331)</sup>The absolute value of the most negative number cannot be represented in two's complement.

which is passed to `mtx_init` to create a mutex object that does not support timeout;

```
mtx_recursive
```

which is passed to `mtx_init` to create a mutex object that supports recursive locking;

```
mtx_timed
```

which is passed to `mtx_init` to create a mutex object that supports timeout;

```
thrd_timedout
```

which is returned by a timed wait function to indicate that the time specified in the call was reached without acquiring the requested resource;

```
thrd_success
```

which is returned by a function to indicate that the requested operation succeeded;

```
thrd_busy
```

which is returned by a function to indicate that the requested operation failed because a resource requested by a test and return function is already in use;

```
thrd_error
```

which is returned by a function to indicate that the requested operation failed; and

```
thrd_nomem
```

which is returned by a function to indicate that the requested operation failed because it was unable to allocate memory.

- 6 [For function pointers that are passed to the functions `call\_once`, `tss\_create`, and `thrd\_create` calls to the underlying function or function literal are sequenced as if they were directly called by the application from the indicated thread.](#)

**Forward references:** date and time (7.27).

## 7.26.2 Initialization functions

### 7.26.2.1 The `call_once` function

#### Synopsis

```
1 #include <threads.h>
   void call_once(once_flag *flag, void (*func)(void));
```

#### Description

- 2 The `call_once` function uses the `once_flag` pointed to by `flag` to ensure that `func` is called exactly once, the first time the `call_once` function is called with that value of `flag`. Completion of an effective call to the `call_once` function synchronizes with all subsequent calls to the `call_once` function with the same value of `flag`.

#### Returns

- 3 The `call_once` function returns no value.

## 7.26.3 Condition variable functions

### 7.26.3.1 The `cnd_broadcast` function